

P2 Documentation and Code Project

You are viewing draft content
created with the use of Claude.AI



P2 Interpreters & Emulators Guide

The XBYTE Engine and Bytecode Dispatch on the Propeller 2

August 2026

Version 1.0.1

Guide Organization

The P2's Hardware Bytecode-Execution Engine

Part I: The Landscape

- Why Emulate on the P2
- What Emulation Asks of You

Part II: XBYTE Fundamentals

- Understanding XBYTE
- The Skip Family
- The Bytecode Stream
- LUT Dispatch

Part III: The XBYTE Engine

- The Dispatch Cycle
- Arming XBYTE
- Table-Size & Compression Modes
- Bytecode Routines
- Debugging XBYTE

Part IV: Building Interpreters and Emulators

- A Minimal Custom VM
- The Three Decisions
- What Will Hurt — A Guest-CPU Survey
- A Tiny CPU Emulator (6502)
- Servicing Guest Interrupts
- Prefixes & Alternate Tables
- XBYTE Beyond Interpreters

Part V: Reference

- Instruction Reference
- Configuration Constants & Patterns

Part VI: Appendices

- A: XBYTE Quick Reference
- B: Instruction Encoding Summary
- C: Further Implementations
- D: Troubleshooting
- Index

Contents

Copyright and License	8
Acknowledgments	8
Sources	9
How to Use This Guide	9
Document Conventions	10
Enhancement Markers	10
 Part I: The Landscape	 11
Chapter 1: Why Emulate on the P2	11
1.1 What draws people to the P2	11
1.2 The range of emulation — and where this book sits	12
1.3 How to read this book	13
Chapter 2: What This Kind of Emulation Asks of You	14
2.1 Where the guest lives — and where it comes from	14
2.2 State, not timing	14
2.3 One guest instruction is often several steps	15
2.4 Guest interrupts are yours to service	15
2.5 When you step off the engine, the in-between work becomes yours	15
2.6 The P2’s memory is a shared budget	16
2.7 The words for all this	16
 Part II: XBYTE Fundamentals	 17
Chapter 3: Understanding XBYTE	17
3.1 The loop at the center of every interpreter	17
3.2 What XBYTE is	18
3.3 Why the P2 has it	18
3.4 The pieces, and where they live	19
3.5 When to reach for XBYTE	19
3.6 What XBYTE costs you	20
3.7 If you’re building...	21
 Chapter 4: The Skip Family	 22

4.1 SKIP — cancel instructions in place	22
4.2 SKIPF — leap the PC over instructions	22
4.3 EXECF — jump, then skip	23
4.4 One body, many bytecodes — the shared-handler idiom	24
4.5 Two things a pattern does when you are not looking	24
4.6 Designing skip patterns — a process	26
Chapter 5: The Bytecode Stream	28
5.1 Priming the FIFO with RDFAST	28
5.2 Fetching bytecodes — RFBYTE	28
5.3 Variable-length operands — RFVAR and RFVARS	29
5.4 The read pointer — GETPTR and PB	29
Chapter 6: LUT Dispatch	30
6.1 The table is 256 EXECF operands in LUT	30
6.2 Building a table entry	31
6.3 The bytecode is handed to you in PA	31
6.4 Dispatch, by hand	31
Part III: The XBYTE Engine	33
Chapter 7: The Dispatch Cycle	33
7.1 The eight clocks	33
7.2 The overhead, exactly	34
7.3 Flags	34
7.4 Interruption — and the fence you will need	34
Chapter 8: Arming XBYTE	36
8.1 One instruction, with \$1FF on the stack	36
8.2 The mode operand	37
8.3 Persistent vs one-shot — SETQ and SETQ2	37
Chapter 9: Table-Size & Compression Modes	39
9.1 Reading the mode operand	39
9.2 Table sizes	40
9.3 Compression — 16 primary plus 240 extended	40
9.4 The F bit — flags from the bytecode	41
9.5 A real mode operand, decoded	42
Chapter 10: Bytecode Routines	43
10.1 The rules a routine follows	43
10.2 The bytecode as an operand — PA	43
10.3 Inline operands — the FIFO and PB	44
10.4 Stopping the engine	45

Chapter 11: Debugging XBYTE	46
11.1 What the hardware will tell you	46
11.2 The debugger shows you the engine's state	47
11.3 The technique the engine cannot give you	47
11.4 Leaving the switch in	48
 Part IV: Building Interpreters and Emulators	 50
Chapter 12: A Minimal Custom VM	50
12.1 The instruction set	50
12.2 The complete program	51
12.3 Folding ADD and SUB into a shared body	52
12.4 In practice: the engine usually runs in its own cog	53
 Chapter 13: The Three Decisions	 54
13.1 Two assets, three decisions	54
13.2 The dispatch ladder	55
13.3 The coupling decision	56
13.4 There is no loop body	57
13.5 Choosing, in order	58
13.6 Why the 6502	59
 Chapter 14: What Will Hurt	 60
14.1 How to read this survey	60
14.2 Can you take the engine?	61
14.3 What will hurt anyway?	62
14.4 Prefixes are two different things	62
14.5 Flags — the cost nobody budgets for	63
14.6 Decimal mode	63
14.7 Guest interrupts	63
14.8 Cycle accuracy	64
14.9 Edge cases	64
14.10 Reading the survey for your own guest	65
 Chapter 15: A Tiny CPU Emulator (6502)	 66
15.1 The mapping	66
15.2 Register file and dispatch	66
15.3 Representative handlers	67
15.4 What this slice shows, and what it omits	67
 Chapter 16: Servicing Guest Interrupts	 69
16.1 The guest's interrupt state is just cog registers	69
16.2 Getting the signal in	69
16.3 Where to poll — the real question	70

16.4 Injecting the interrupt	71
16.5 Halt, and waiting for an interrupt	72
16.6 What this costs you	72
Chapter 17: Prefixes and Alternate Tables	74
17.1 Prefixes are two different things	74
17.2 Map prefixes — the 6809's pages	75
17.3 Modifier prefixes — state, not dispatch	75
17.4 SETQ2 is bigger than prefixes	76
Chapter 18: XBYTE Beyond Interpreters	78
18.1 Two assets, three widening features	78
18.2 The application map	79
18.3 A terminal reader — the table as state	80
18.4 A MIDI dispatcher — the byte as data	80
18.5 A display list — the stream as a movable cursor	81
18.6 SETQ2 is a general mode switch	83
18.7 When XBYTE is the wrong tool	83
Part V: Reference	86
Chapter 19: Instruction Reference	86
19.1 The skip family	86
19.2 Arming	87
19.3 The FIFO bytecode stream	87
Chapter 20: Configuration Constants & Patterns	88
20.1 The mode operand	88
20.2 Registers and ranges	88
20.3 The arming pattern	89
Part VI: Appendices	90
Appendix A: XBYTE Quick Reference	90
Appendix B: Instruction Encoding Summary	92
Appendix C: Further Implementations	93
C.1 The reference: Parallax's own XBYTE	93
C.2 P2 Arc8de — eight 8080 arcade machines on one P2	94
C.3 8080 games emulators — XBYTE in production	94
C.4 The “Yume” emulator suite — console emulators on P2 + PSRAM	94
C.5 Intel 8086 — the same guest, more than one way	95
C.6 Zog — the ZPU	95

C.7 riscvemu — the road not taken	95
C.8 A minimal community example — the “essential XBYTE” VM	95
Appendix D: Troubleshooting	97
Index	99

List of Figures

3.1	The XBYTE dispatch loop: fetch, look up, dispatch, run, repeat	18
6.1	LUT dispatch-table entry: one EXECF operand	30
7.1	The 8-clock XBYTE dispatch cycle (clock 1 overlaps the prior _RET_)	34
9.1	The mode operand %A000000xF (256-entry table form)	39

Copyright and License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose, even commercially

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made.
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

To view the full license, visit: <https://creativecommons.org/licenses/by-sa/4.0/>

Trademarks

Parallax, Propeller, Spin, and the Parallax logo are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC or Parallax Inc.

Acknowledgments

This guide would not exist without the contributions of many individuals and organizations:

Parallax Inc. for creating the Propeller 2 microcontroller and providing the comprehensive reference documentation that forms the foundation of this work.

Chip Gracey for the design of XBYTE, the SKIP family, and the LUT/FIFO hardware, and for maintaining the detailed silicon documentation that defines their behavior.

The P2 Community for the interpreters, virtual machines, and CPU emulators whose real-world use proves what the engine makes possible.

Sources

This guide draws on the following primary sources:

- **Parallax Propeller 2 Documentation v35 - Rev B/C** (Chip Gracey, Parallax Inc.) — the XBYTE dispatch cycle, overhead figures, table-size and compression modes, and the F bit
- **P2 Knowledge Base YAML** (Iron Sheep Productions / P2 Knowledge Base Project) — the XBYTE engine reference and the per-instruction encodings for the SKIP family, the FIFO read instructions, and SETQ/SETQ2

How to Use This Guide

This guide serves developers building interpreters, virtual machines, and CPU emulators on the Propeller 2. It assumes familiarity with P2 cog/hub/LUT architecture, basic PASM2, and the hub FIFO (RDFAST and the RFxxxx reads).

Structure:

- **Part I (The Landscape)** builds the picture before any mechanism — what emulation is, why the P2 is unusually good at it, and the handful of concerns any emulator must handle. It names no P2 machinery. Read it first; skim it if you have built emulators before.
- **Part II (Fundamentals)** builds the mental model of the engine — it teaches the SKIP family (SKIP/SKIPF/EXECF) before the XBYTE engine, because the engine is built out of them.
- **Part III (The Engine)** is the reference for the dispatch cycle, arming, the table-size and compression modes, the rules bytecode routines follow, and how to debug a hardware dispatch loop.
- **Part IV (Building Interpreters and Emulators)** proves the engine by building — a minimal custom VM — then steps back to the decisions that come *before* any emulator: which of the engine's assets you can take, what each classic guest CPU will cost you, and how to service guest interrupts and prefix bytes. It closes by widening the frame beyond interpreters entirely.
- **Part V (Reference)** is quick lookup for the instructions and the configuration bits.
- **Appendices** contain quick-reference cards, the encoding summary, pointers to community implementations, and troubleshooting.

The 6502 emulator in Part IV is deliberately **tiny and illustrative** — enough to show the technique end to end, not a faithful or complete emulator. The guide's two complete programs — the minimal VM and the display-list engine — do compile.

Document Conventions

Element	Format	Example
Instructions	Bold uppercase	EXECF, SETQ
Registers / symbols	Monospace	PA, PB, _RET_
Bit fields	Brackets	D[31:10], D[9:0], b[7:4]
Binary	Percent + underscores	%A0000000xF
Hexadecimal	Dollar prefix	\$1F6, \$1FF

Enhancement Markers

Three colored callout boxes set “things to know” apart from the running text:

- **CAUTION** (amber) — common mistakes with non-obvious consequences
- **TIP** (teal) — non-obvious techniques or optimizations
- **HARDWARE** (graphite) — silicon-level details affecting usage

Part I: The Landscape

This Part is the map. It says what emulation is, why the Propeller 2 is unusually good at it, and the handful of things any emulator of the kind we build must reckon with — all before a single P2 instruction is named. Read it and you will meet the engine in the next Part already knowing what you are pointing it at, and why. If you have built emulators before, skim it; if you have not, it is the ground the rest of the book stands on.

Chapter 1: Why Emulate on the P2

1.1 What draws people to the P2

Three things draw people to the Propeller 2 for this work — and raw speed is not the first of them.

The P2 is built for interpreters. Underneath every interpreter is one small loop: fetch the next instruction, look up what it means, carry it out, repeat. The P2 has hardware made for exactly that loop, so writing an interpreter — for a wide range of instruction sets — is unusually direct, with the machine carrying the busywork. This flexibility is the largest part of the draw: you can turn a stream of *someone else's* instructions into running code, and the P2 was designed to help you do it. (The P2 doing the emulating is the **host**; the processor it reproduces — an 8080, a 6502, a Z80 — is the **guest**. Those two words run through the whole book. You meet the hardware itself in Part II.)

A whole machine on one chip. The P2 has eight processors, called *cogs*. One cog can run the emulator; the seven beside it can generate the video and sound the original needed separate hardware to produce — a whole late-1970s arcade machine, processor and display and audio, on one inexpensive chip. Speed lives here too, honestly sized: the host runs its own instructions far faster than the one-to-a-few-megahertz classics, so for many **low-end** guests — simple 8-bit micros, only a few steps to reproduce each instruction — it keeps up with room to spare, sometimes beating the original outright. That is a bonus simple guests hand you, not a promise for all of them. The more complex a guest's instructions get — several bytes each, many addressing modes, several pieces of state touched at once — the more host steps every one costs, and the smaller that margin grows, until simply *keeping* real time is the goal. (Chapter 2 returns to this: how many steps a guest instruction takes is one of the first things that decide what it will cost you.)

More than one at a time. Those eight cogs are independent, so a single chip can run several emulators side by side — different guests, or many copies of the same one — each on its own cog,

all at once. On most platforms an emulator is the whole machine's job; on the P2 it is often a single cog's job, and the rest of the chip stays free.

If you have done this before: the surprising part is usually not the CPU — it is how much the chip does *around* it: hardware that carries the dispatch, sibling cogs that drive the screen and sound, room for more than one guest at once. On the P2, the emulator is often not the hard part of the project.

1.2 The range of emulation — and where this book sits

Emulation is a broad word, and it helps to see the whole field before we narrow to our corner of it. Here is the range, from the most exacting to the most practical — with a plain note on each, and on why we do or do not build it.

- **Cycle-accurate (hardware) simulation.** Reproduces not only *what* the machine computes but *when* — clock tick by clock tick — so timing-dependent tricks, and even glitches, behave exactly as on the original. It is the standard for faithful preservation and the most expensive kind to build. **This book does not do this**, and later — in the guest-CPU survey (Chapter 14) — you will see *why* the P2's fastest way of running an emulator and exact-timing accuracy pull in opposite directions.
- **Full-system emulation.** The guest's processor *together with* its peripherals, timers, and buses, modeled as one whole machine. **We stay at the processor.** Its peripherals you build yourself, out of the P2's own I/O — which on this chip is a strength, not a shortfall.
- **Dynamic binary translation (JIT).** Rather than read and carry out each guest instruction every time it runs, translate whole blocks of guest code into host code *once*, then run the translation. Fast on hot loops; considerably more complex to build. **We interpret; we do not translate.** It is named here so you know the road exists — the closing chapter (Chapter 18) notes where it would branch off.
- **Static recompilation.** Translate the entire guest program to host code *ahead of time*, offline. **Out of scope** — that is a compiler project, not a runtime, and a different book.
- **Instruction-at-a-time interpretation — reproducing behavior, not timing.** ← **This is the book.** You reproduce what the guest *computes*: the contents of its registers, its status flags, its memory, and the path its control flow takes — one guest instruction at a time. You do **not** reproduce exact timing. For the very large range of things people actually want to run, that is precisely enough — and it is the kind the P2's hardware was built to make cheap.

This framing serves whether or not emulation is new to you. If it is new, you now know the shape of the territory and where we are standing in it. If it is not, you know exactly which trade-offs we are *not* making.

1.3 How to read this book

This guide is written for two readers, and it is one road with two on-ramps.

If emulation is new to you: read this Part straight through, then continue in order. It builds the picture — what emulation is, why the P2 suits it, and what any emulator must handle — that every later chapter assumes. You will reach the P2’s dispatch engine already understanding what it is *for*, instead of gathering its details and hoping they add up.

If you have built emulators before: skim the field above, note that we work **at the instruction level** and reproduce **behavior rather than timing**, and go to the engine (Part II) and then to the decisions chapter (Chapter 13), where the P2-specific judgment lives: *which* of the engine’s assets your particular guest can actually use. For many guests the answer is only one of them, not both — and Chapter 13 is where that is worked out.

The next chapter meets you in the middle: it lays out, in plain terms, the handful of concerns every emulator of our kind must answer — and then, at its end, gives you the vocabulary the rest of the book will speak.

Chapter 2: What This Kind of Emulation Asks of You

You have chosen a lane: reproduce the guest’s *behavior*, one instruction at a time, and let exact timing go. That single choice commits you to a short list of questions — the same questions every emulator of this kind must answer, whatever the guest. Meet them here, in plain language and without any P2 machinery, and carry each one forward as a question about *your* guest. The rest of the book is where each is answered in full.

2.1 Where the guest lives — and where it comes from

This is the first fork, and it decides more than any other.

Where the guest runs from. Some guests are small enough that their entire memory — program and data together — fits in the P2’s fast on-chip memory. Others are too large and must live in slower, larger external memory. Which world you are in sets how fast your emulator can run *and* how big a machine you can attempt at all. It is worth settling before you write a line, because it shapes everything after it.

Where the guest comes from. A guest’s program — and the data it needs, its ROMs, its media, its disk or cartridge images — has to be *loaded* from somewhere into that run-from memory before anything executes. On the P2 that somewhere can be on-board flash, a host connection, or a microSD card; on the boards with a card slot, a single card can hold a whole *library* of programs and assets to load on demand. Where things come from and where they run are two separate decisions, and they interact: a card feeds a small guest into fast memory or a large guest into external memory just the same.

One thing we deliberately do *not* do: page a too-large guest’s code in and out of fast memory *as it runs*. When a guest is too big for on-chip memory, the P2’s answer is to hold it, whole, in external memory — the decisions chapter (Chapter 13) explains why holding it whole beats shuffling it in and out, and it is the right instinct to carry.

Carry the question: does my guest fit in fast memory, or must it live in external memory — and how does it, and its assets, get loaded there?

2.2 State, not timing

You reproduce what the guest *computes* — the values in its registers, its status flags, its memory, the branches it takes — but not the exact number of clock-ticks each step consumed. This is the lane you chose, and it is the right one for almost everything, because matching *state* is what makes programs run correctly, while matching *timing* exactly is a separate and far costlier project. A few guests carry software that leans on precise timing; most do not.

Carry the question: does my guest have anything that depends on exact timing — and if it does, how much of it must I honor?

2.3 One guest instruction is often several steps

A single instruction on the guest is rarely a single step to reproduce. It may be followed in the stream by operand bytes it needs. It may belong to a family whose members share most of their work. On some guests an instruction is introduced by an extra byte ahead of it that changes what it means. So “carry out one guest instruction” can unfold into: fetch it, fetch what follows, decide what it really is, then act. How much this costs you depends entirely on how *regular* your guest’s instructions are.

Carry the question: how regular is my guest’s instruction shape — and how many of its instructions are variations on a common pattern?

2.4 Guest interrupts are yours to service

Real machines get interrupted — a key is pressed, a timer expires, a scan line finishes — and the guest’s own software expects those interruptions, and expects them to arrive the way its hardware delivered them. Nothing about faithfully running the guest’s *ordinary* instructions handles this; you arrange it yourself, deliberately, as a thing the emulator does between the guest’s instructions.

Carry the question: does my guest depend on interrupts — and how faithfully must I reproduce when and how they arrive?

2.5 When you step off the engine, the in-between work becomes yours

The P2 has a hardware feature — you meet it in the next Part — that runs the fetch-and-dispatch loop of an interpreter *for you, in hardware*, at a small fixed cost per instruction. It comes with one catch worth planting now: it runs the loop so completely that it leaves **no gap** between one guest instruction and the next. And that gap is exactly where a real emulator often needs to slip in small recurring jobs — pacing itself to real speed, checking whether an interrupt has arrived, ticking a counter the guest can read back, leaving a hook for a debugger.

So there is a trade at the center of this whole book: take the free loop and you give up the one place that in-between work naturally lives; keep a place for that work and you run the loop yourself, at a small cost. Which is the bargain depends on how much your guest needs done between its instructions — and that is a decision you make per project, with eyes open.

Carry the question: how much work does my guest need done between its instructions?

2.6 The P2's memory is a shared budget

The fast memory and the large memory your emulator wants are wanted by the rest of your system too — most often by the very display that shows the emulated machine's screen. On the P2 board built for the biggest jobs, the same external memory that could hold a large guest is also where a high-resolution display keeps its picture, and both reach it through one shared pipe. Memory on this chip is not a single pool you draw from freely; it is a budget that competing jobs share.

Carry the question: what else on my P2 wants the memory my emulator needs — and can they share it?

2.7 The words for all this

The ideas are now in place, so here are the names the field puts on them — the vocabulary the rest of the book speaks, gathered as one page to turn back to. You already met **host** and **guest** in Chapter 1; they lead the list, and the rest join them here.

- **Host** — the P2 itself: its hardware and its native instruction set, the machine doing the emulating.
- **Guest** — the processor you are emulating, together with its programs. A guest's compiled program is its **object code** (or **guest binary**) — the machine code you load and carry out.
- **Emulator** — a program that makes the host behave as the guest, so that guest object code runs on it. **Interpreter** — an emulator that works by reading and carrying out the guest's instructions one at a time, as opposed to *translating* them ahead of time. This book builds instruction-interpreting emulators, which is why the two words travel together here.
- **Behavior-accurate** (also called *architectural* accuracy) — reproducing the guest's architectural state: its registers, its flags, its memory, and its control flow. **Cycle-accurate** — reproducing, in addition, the exact timing, clock for clock. This book is behavior-accurate; it is not cycle-accurate, and that is a deliberate choice, not a limitation you will trip over later.
- **Emulator core** (or **CPU core**) — the host code that does the fetching, decoding, and carrying-out of guest instructions: the heart of the emulator, and the thing the rest of this book teaches you to build well.

With the picture drawn and the words in hand, the next Part is where the P2 earns its reputation — the hardware that runs the interpreter's loop — and, a few chapters on, the decision of how much of that hardware your particular guest should use.

Part II: XBYTE Fundamentals

This part builds the mental model of the engine. It opens on the one idea XBYTE exists to serve — the loop at the center of every interpreter — then teaches the **skip family** (SKIP, SKIPF, EXECF) that XBYTE is built from, the **FIFO** that feeds it bytecodes, and the **LUT dispatch table** it reads. By the end of Part II, XBYTE itself (Part III) is no longer magic: it is hardware that runs a dispatch you already understand, once per bytecode, at a fixed six-clock cost (§7.2).

Chapter 3: Understanding XBYTE

3.1 The loop at the center of every interpreter

Every interpreter ever written runs the same loop. Fetch the next instruction — a small number, a *bytecode* — from a stream. Look up what that number means. Jump to the code that does it. Run that code. Repeat.

A *bytecode* is just one small number that stands for one operation the interpreter knows how to perform: “push a constant,” “add the top two stack items,” “jump if zero.” A program, in this style, is a stream of those numbers. The interpreter’s whole job is to walk the stream and, for each number, run the matching *routine* (also called a *handler*).

Written by hand on the P2, that loop costs something on every single bytecode: read the byte, use it to index a table, branch to the handler. Those few instructions run *between* every pair of useful operations, so their cost is multiplied by the length of the program. For an interpreter, the dispatch loop is the tax you pay on everything.

3.2 What XBYTE is

XBYTE is hardware that runs that loop for you. Once armed, the P2’s bytecode-execution engine fetches the next bytecode from the hub FIFO, writes it where your handler can see it, indexes a 256-entry dispatch table in LUT RAM, and jumps to the handler — and when the handler ends, it does it all again, automatically, until you stop it. You write the handlers; the engine runs the loop between them.

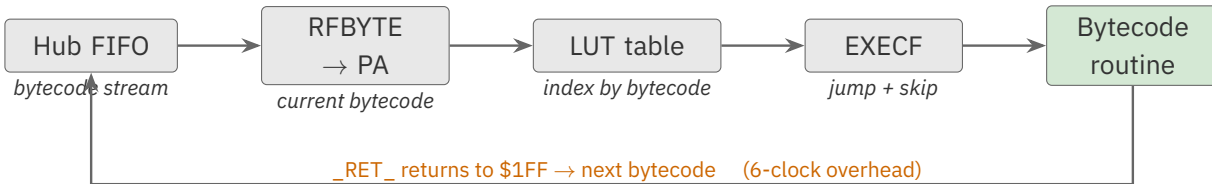


Figure 3.1. The XBYTE dispatch loop: fetch, look up, dispatch, run, repeat

The payoff is the cost of that loop. The *Parallax Propeller 2 Documentation v35* states the overhead of XBYTE dispatch is **6 clocks per bytecode**, “including `_RET_` at the end of each bytecode routine” — against the **9 clocks** the same fetch-look-up-execute sequence costs in software (“2+3+4, or 9, clocks to get the next bytecode, look it up, then execute that bytecode’s routine”). A bytecode routine “could be as short as a single 2-clock instruction with a `_RET_` prefix, making the total XBYTE loop take only 8 clocks.”

HARDWARE

XBYTE is built out of ordinary instructions you can use yourself. The engine’s dispatch is an **EXECF** — a jump plus a SKIP pattern — fed from a **RDLUT**, fed from an **RFBYTE** off the FIFO. None of these are special to XBYTE. Chapter 4 teaches them as the everyday instructions they are; Chapter 7 shows the engine running exactly that sequence, in hardware, in six clocks.

3.3 Why the P2 has it

The P2 is fast at running native PASM2, but native code is large: a big program does not fit in a cog’s 512 longs, and even hub-executed code trades speed for space. Interpreted bytecode is the classic answer — compact programs, a small interpreter — and it is how Spin2 itself runs on the P2. The cost of interpretation is the dispatch loop, and XBYTE exists to make that cost small enough that an interpreted language, or an emulated CPU, stays practical.

That second case is the one this guide builds toward. Emulating another processor *is* interpretation: each of the guest’s instructions is a “bytecode,” and the handler is the PASM2 that reproduces it. When dispatch is cheap, a single cog can emulate a whole small CPU and still have clocks left to drive video and sound — which is what the 8080 arcade emulators in Appendix C achieve.

CAUTION

Emulating a CPU is where using XBYTE becomes conditional. Several of the P2’s console emulators (Appendix C) do **not** use the engine at all, for reasons that have nothing to do with their guests’ instruction sets. Chapters 13 and 14 set out those reasons; read them before you commit to an architecture.

3.4 The pieces, and where they live

XBYTE coordinates six pieces of the P2 you already have. This guide covers each in the chapter shown; here is the map.

Piece	What it does for XBYTE	Where it is	Chapter
The hub FIFO	delivers the bytecode stream, one byte at a time	hub → cog, primed by RDFAST	5
The dispatch table	maps each bytecode to a handler + a skip pattern	256 longs in LUT RAM	6
EXECF	the jump-plus-skip that <i>is</i> dispatch	a cog instruction	4
PA (\$1F6)	holds the current bytecode for the handler to use	a cog register	6
PB (\$1F7)	holds the FIFO pointer (for inline operands)	a cog register	5
The hardware stack	holds \$1FF, the address each routine returns <i>to</i>	the cog’s call stack	8

The handlers themselves live in cog or LUT RAM and end in **RET** or **_RET_**. The engine is armed with one instruction — **SETQ** (or **SETQ2**) — and a \$1FF on the stack. Those are Chapter 8.

3.5 When to reach for XBYTE

XBYTE pays off when dispatch cost dominates — any time your “program” is a stream of byte-sized operations **read from hub memory**, and each one selects a small piece of work. The famous case is an interpreter or VM with many small operations, or a CPU emulator, but it is not the only one: a protocol parser, a data-format decoder, a graphics display list, an event sequencer — anything shaped like *walk a byte stream, dispatch on each byte* is a candidate. Chapter 18 widens the lens well beyond interpreters.

It is the **wrong** tool when there is no stream to walk, or when the “dispatch” is really just a lookup with no per-symbol work to do — a plain RDLUT is cheaper than arming the engine. And there are three conditions that rule it out entirely, no matter how well your problem fits in every other respect. They are worth knowing now, in one sentence each, because they are architectural:

- **Your stream must live in hub RAM.** The FIFO reads hub, and nothing else. Data in external PSRAM cannot be auto-fetched — at all.
- **LUT must be free.** The engine reads its table from LUT, so if a palette or a buffer has claimed it, the engine is unavailable.
- **You must have nothing to do between symbols.** The engine’s loop is *hardware* — there is no loop body, and therefore nowhere to put per-symbol work like pacing, tracing, or a progress check.

None of these is a matter of degree. Chapter 13 is where they come from and §18.7 is the full list; if one of them holds, you want the software loop of §6.4 — which is not a consolation prize, but what most working P2 emulators actually ship.

TIP

If you have written an interpreter before: XBYTE replaces your `next:` dispatch label — the `fetch / index / jump` you wrote by hand — with hardware. Your handlers stay yours; you delete the loop between them — including anything else you were keeping there.

3.6 What XBYTE costs you

The engine is not free, and its price is paid in cog resources rather than clocks. Know the bill before you commit:

Resource	What arming XBYTE takes
LUT RAM	256 longs for the dispatch table — half of LUT. Smaller table modes cost less (Chapter 9)
Cog / LUT space	your handlers, which must live in cog or LUT — they cannot run from hub
The hardware stack	one of its eight levels, holding \$1FF, for as long as the engine runs (§8.1)
PA (\$1F6)	overwritten with the current bytecode on every dispatch
PB (\$1F7)	overwritten with the FIFO read pointer on every dispatch
The cog’s FIFO	held by RDFAST for the bytecode stream — so it cannot simultaneously stream video or drive a block move
The dispatch loop	there isn’t one. No place for per-bytecode work of any kind

The first six are ordinary budgeting. The last one is different in kind — not a resource the engine spends but a place to work that it removes — and it is the subject of §13.4.

3.7 If you're building...

You have probably arrived with an application already in mind. Find it here; the chapters on the right are the ones to read closely first.

If you're building...	XBYTE gives you...	Start at
a bytecode VM or scripting language	the whole engine — this is what it was built for	Ch. 12
a CPU emulator	it depends on your guest — often one asset, not both	Ch. 13, Ch. 14
a terminal / ANSI parser	the <i>table</i> as state — ESC borrows an alternate table	§18.3
a MIDI or protocol decoder	the <i>byte</i> as data — the channel or type rides in PA	§18.4
a graphics display list	the <i>stream</i> as a movable cursor	§18.5
a binary format / TLV decoder	the byte as a type tag	§18.2
an event or animation sequencer	seek — the read cursor loops and branches for free	§18.2
something else that walks a byte stream	the general test	§3.5, then §18.7

If your project is a **CPU emulator**, read Chapters 13 and 14 before you write a line. They will tell you which of the engine's two assets you can actually take — and for a good number of guests, the honest answer is *one of them*.

To see what the engine makes possible on real silicon — and, just as usefully, where working emulators have chosen *not* to use it — see **Appendix C: Further Implementations**.

Chapter 4: The Skip Family

XBYTE is built out of three related instructions: **SKIP**, **SKIPF**, and **EXECF**. They are useful on their own, and understanding them is the whole secret to understanding the engine — because XBYTE’s dispatch is an EXECF, and its compact handlers are built with SKIPF. This chapter teaches the family first, as ordinary instructions. The engine in Part III then needs almost no new ideas.

All three take a pattern of bits and use it to *not execute* selected instructions. The difference is in *how* they SKIP and *what else* they do.

4.1 SKIP — cancel instructions in place

SKIP takes a 32-bit pattern in D and, as the next up-to-32 instructions come down the pipeline, **cancels** each one whose corresponding bit is set. Bit 0 governs the first instruction after SKIP, bit 1 the second, and so on. A cancelled instruction still passes through the pipeline — it simply has no effect, taking its time but doing nothing.

SKIP	#%0000_0110	' cancel the 2nd and 3rd
		' following instructions
ADD	x, #1	' bit 0 = 0 -> runs
ADD	x, #2	' bit 1 = 1 -> cancelled
ADD	x, #4	' bit 2 = 1 -> cancelled
ADD	x, #8	' bit 3 = 0 -> runs

x ends up holding %1001 — read it as a bitmap of which instructions survived.

Because a cancelled instruction still spends its clocks, SKIP’s cost is the cost of the instructions it skips over. Its value is that one straight-line block of code can be made to behave like many different sequences, chosen by the pattern — the basis of a *shared handler*.

HARDWARE

SKIP works even in hub-executed code. It is the general-purpose member of the family — it stays in the normal execution flow and cancels, rather than jumping. That makes it the one to reach for in HUBEXEC, where SKIPF’s PC-leap does not apply.

4.2 SKIPF — leap the PC over instructions

SKIPF is the *fast* SKIP. Instead of cancelling instructions one by one as they arrive, it makes the program counter **leap past** the skipped instructions entirely, in cog or LUT RAM. Where SKIP pays for what it skips, SKIPF skips for free — a run of skipped instructions costs essentially nothing, because the PC simply does not visit them.

```

SKIPF    #%0000_0110      ' leap over the 2nd and 3rd
ADD      x, #1             ' runs
ADD      x, #2             ' leapt over (no cost)
ADD      x, #4             ' leapt over (no cost)
ADD      x, #8             ' runs

```

The trade for that speed is the restriction: SKIPF works in **cog and LUT RAM only** (the PC-leap needs the cog/LUT addressing). Its pattern is the full **32 bits** of the operand D, applied LSB-first — so a standalone SKIPF governs the next 32 instructions. (Inside XBYTE the pattern comes from **EXECF**, which spends its low 10 bits on a jump address and so carries only a **22-bit** pattern — §4.3. That 22 is the width XBYTE stores in each dispatch-table entry: not because SKIPF is 22-bit, but because EXECF reserves ten bits for *where to jump*.)

HARDWARE

“Free” has one small print. SKIPF steps the PC forward 1 to 8 instructions at a time, so **at most 7 in a row are leapt at once**; every **8th** consecutive skipped instruction is stepped through as a **2-clock NOP**. A handler that skips fewer than eight in an unbroken run — the usual case — really does SKIP for free; only a long unbroken run pays the occasional 2-clock tick.

4.3 EXECF — jump, then skip

EXECF is SKIPF with a jump bolted on. Its single operand D carries both:

- **D[9:0]** — a 10-bit cog/LUT address to **jump to**
- **D[31:10]** — a 22-bit **SKIPF pattern** to apply once there

So EXECF means: *go to this routine, and skip these instructions inside it*. One instruction, one operand, and you have selected both *which* code to run and *which variant* of it.

That is the entire mechanism of dispatch. If a table entry holds an EXECF operand — a handler address in the low 10 bits, a SKIP pattern in the high 22 — then “execute the operand” is exactly “run handler N in variant M.” XBYTE’s dispatch table is precisely a table of EXECF operands (Chapter 6), and the engine’s core step is precisely an EXECF (Chapter 7).

That operand is just one long — an address ORed with a pattern shifted up by ten. Nothing more constructs it:

```

' Handler at cog address $120; pattern %1110 keeps instruction 0
' and skips the next three (a set bit skips; §4.2).
entry          LONG    $120 | (%1110 << 10)      ' "run one line of four"

```

Instruction	Skips by	Also does	Pattern width	Works in
SKIP	cancelling in place	—	32 bits	cog, LUT, hub
SKIPF	leaping the PC	—	32 bits	cog, LUT
EXECF	leaping the PC	jumps to D[9:0] first	22 bits (D[31:10])	cog, LUT

4.4 One body, many bytecodes — the shared-handler idiom

The reason the SKIP family matters to an interpreter is *code sharing*. Many bytecodes differ only slightly — “add,” “subtract,” “and,” “or” might be one routine that loads two operands, performs one ALU operation, and stores the result, where only the middle instruction changes. Rather than write four routines, write one body containing all four ALU operations and give each bytecode a SKIP pattern that leaves *its* operation and skips the other three.

```
' One shared body for several two-operand ALU bytecodes.
' Each bytecode's dispatch entry supplies a SKIPF pattern that
' leaves exactly one of the four ALU lines and skips the rest.
alu_body
    CALL    #pop_two      ' operands -> a, b
    ADD     a, b          ' "ADD" bytecode keeps this
    SUB     a, b          ' "SUB" bytecode keeps this
    AND     a, b          ' "AND" bytecode keeps this
    OR      a, b          ' "OR"  bytecode keeps this
    _ret_   CALL    #push_a  ' result -> stack, return
```

The bytecode that means “subtract” points at `alu_body` with a SKIP pattern that leaps the ADD, AND, and OR lines; “add” skips the other three; and so on. Four bytecodes, one body. This is the most common XBYTE handler pattern, and it is pure SKIPF — the engine just supplies the pattern for you, from the table, on every bytecode.

TIP

Design handler bodies so the *common* path has the fewest skips. Every kept instruction runs; a skipped one costs almost nothing (§4.2). A well-factored shared body can serve a dozen related bytecodes with one copy of the code.

4.5 Two things a pattern does when you are not looking

The shared body above depends on one behaviour and must step around one trap. Both matter before you write your own.

Skipping is suspended inside a CALL.

Look at `alu_body` again. It opens with `CALL #pop_two` and ends with `_ret_ CALL #push_a`. Those helpers contain instructions of their own — and the bytecode’s SKIP pattern is **not** applied to them. The P2 suspends skipping for the duration of a call and resumes it on return.

This is not folklore; the hardware tracks it explicitly. The **CALL depth since the pattern began** is one of the fields GETBRK reports (§11.1), and **skipping is suspended whenever that depth is non-zero**.

It is also the fact that makes shared bodies *practical*. Without it, every SKIP pattern would have to account for every instruction inside every helper the body calls — and factoring common work into subroutines would be impractical. You have been relying on it since `alu_body` above.

A pattern longer than its body runs past the end of it.

The pattern is consumed as instructions execute. If it carries more bits than the body has instructions, the leftovers do not evaporate. They fall on **whatever runs next**.

Under XBYTE this is harmless — the engine cancels any leftover pattern at clock 1 of the next dispatch (§7.1). But it becomes a real trap the moment you dispatch by hand (§6.4, and Chapter 11, where it bites hardest). The discipline that prevents it is one line long: **size each pattern to the body it belongs to**.

CAUTION

A skip pattern counts *instructions*, not source lines — and `##` makes one line into two instructions.

A large immediate does not fit in an instruction’s 9-bit operand field, so the assembler quietly emits an **AUGS** ahead of it to supply the missing bits. One line of source, **two longs of code**:

ADD	x, #100	' 1 instruction
ADD	x, ##1000	' 2 instructions: AUGS, then ADD

Now count what that does to a hand-written pattern. If a skipped body contains a `##` operand — a large constant, a hub address, a `##`-form jump target — then **every bit of your pattern from that line onwards is off by one**, and the symptom is that the wrong instructions run for reasons the source code does not show.

Two defences, and you want both. **Keep large constants out of skipped bodies** (load them into a register before the pattern begins). And when a pattern misbehaves, **count the longs, not the lines** — the debugger’s strikethrough view (§11.2) shows you the instructions the hardware actually sees, which is exactly the view you need.

HARDWARE

There is a third consequence, and it hands you a spare bit.

A dispatch-table entry is `handler | pattern << 10` (§6.1). So **bit 10 is the pattern's lowest bit — the bit that would cancel the handler's first instruction**. But you jumped there in order to run that instruction. A normal dispatch entry therefore leaves bit 10 clear.

Which makes bit 10 spare storage: **one free boolean per bytecode**. Read it and clear it in a single instruction on the way in, and EXECF still sees a clean pattern:

```

        BITL    entry, #10      wcz ' C,Z = old bit; bit cleared
if_c    JMP     #special_case   ' pattern clean for EXECF

```

This carries a per-opcode flag that would otherwise have cost a second table.

4.6 Designing skip patterns — a process

The shared-handler idiom is powerful, but a body serving a dozen bytecodes is only as good as the patterns that carve it up. Here is a repeatable way to design them — and, at the end, the two ways a pattern misbehaves for reasons the source does not show.

The process.

1. **Find the family.** Group the bytecodes that do *almost* the same work — same operands fetched, same result stored, differing only in the operation between. An ALU group, a load-/store group, a “push small constant” group. The wider and more regular the family, the more one body earns its keep.
2. **Write the superset body.** Lay every instruction *any* member needs into one straight-line body, in a fixed order. Each member is then a *subset* of that body — its pattern leaves that member's instructions and skips the rest (§4.4).
3. **Factor shared work into calls.** Common setup and teardown — pop the operands, push the result — go in subroutines. Skipping is *suspended inside a call* (§4.5), so a helper's instructions never consume pattern bits; the body stays short and every pattern stays simple. This is what makes wide families practical at all.
4. **Assign each member's pattern, common path first.** A member's pattern skips the instructions it does not want. Order the body so the *most common* members skip the least — under SKIPF every kept instruction runs and every skipped one is free (§4.2). Build the table entry with the handler address in the low 10 bits and the pattern in the high 22 (§6.2).
5. **When a pattern can't say it, use the flag.** A SKIP pattern chooses *which instructions* run; it cannot make a kept instruction *behave two ways*. When members differ in behaviour rather than in instruction-selection — a conditional, a loop count, two variants of one operation — carry two selector bits in the bytecode and let the **F bit** deliver them as flags (§9.4). Pattern and flag are complementary selectors; reach for the flag only when the pattern runs out.

Two ways a pattern misleads you (both from §4.5): a *##* immediate is *two* longs, so one *##* inside a skipped region throws every bit after it off by one; and a pattern with more bits than its body has instructions spills onto whatever runs next. Count longs, not lines, and size each pattern to its body.

Avoid.

- Transposing the fields — address is the **low** 10 bits, pattern the **high** 22 (§6.2).
- Letting a family outgrow the **22-bit window**: the dispatch pattern is 22 bits, so a body can be skipped across at most 22 instructions. A wider family splits into two bodies, or pushes more work into calls.
- Accounting for a helper's internals in a pattern — you never need to (§4.5), and it breaks the moment the helper changes.

Get the family and the body right and each pattern is just *keep these, skip those*, counted in longs.

Chapter 5: The Bytecode Stream

XBYTE reads its bytecodes from the **hub FIFO** — the same fast, sequential hub-reading hardware every cog has. This chapter covers how the stream is primed and read, because a bytecode program is just bytes in hub memory, and the FIFO is how they reach the engine.

5.1 Priming the FIFO with RDFAST

Before any bytecode can be fetched, the cog’s FIFO must be pointed at the bytecode stream in hub memory. **RDFAST** does that: it begins a fast, sequential hub read starting at a given address.

```
RDFAST  #0, ##program      ' FIFO now streams bytes
                                ' from 'program' onward
```

- **S** holds the hub start address (S[19:0]).
- **D** holds the block size in 64-byte units (D[13:0]); **0 means “no limit”** — the FIFO streams continuously, which is what an interpreter usually wants. D[31] is a no-wait flag.

After RDFAST, the FIFO delivers bytes in order, refilling from hub memory on its own.

CAUTION

Arm the FIFO before arming the engine. XBYTE’s very first action is an **RFBYTE** off the FIFO (Chapter 7). If RDFAST has not pointed the FIFO at your bytecode stream, that first fetch reads undefined data and the interpreter dispatches garbage. RDFAST first, then start XBYTE.

5.2 Fetching bytecodes — RFBYTE

RFBYTE reads the next byte from the FIFO, zero-extended, into D. It is the instruction XBYTE issues to fetch each bytecode.

```
RFBYTE  bytecode          ' next stream byte -> bytecode
```

RFBYTE is a 2-clock instruction. It optionally sets flags (C = the byte’s MSB, Z if the byte is zero), though in XBYTE dispatch the engine manages flags itself (Chapter 9). Its companions **RFWORD** and **RFLONG** read a word or a long from the stream the same way — useful inside a handler that needs a fixed-size *inline operand* following the bytecode.

5.3 Variable-length operands — RFVAR and RFVARS

Many bytecode formats follow an operation with an operand whose size varies — a small value takes one byte, a larger one takes more. The FIFO reads these directly:

- **RFVAR** reads a **zero-extended** 1-to-4-byte variable-length value into D (C is cleared).
- **RFVARS** reads a **sign-extended** 1-to-4-byte variable-length value into D (C = the value's MSB).

```
' Inside a "push constant" handler: the constant follows the
' bytecode in the stream as a variable-length value.
push_const
           RFVAR   value           ' pull the inline constant
      _ret_  CALL   #push_value     ' onto the VM stack, return
```

Because the FIFO advances as it is read, a handler can pull as many inline operand bytes as the operation needs and the next RFBYTE — the next dispatch — naturally lands on the following bytecode. The stream stays self-describing: operations and their operands interleave, and the read position takes care of itself.

TIP

Use **RFVAR** for unsigned operands (addresses, indices) and **RFVARS** for signed ones (relative branches, signed immediates). Choosing the right one means the value arrives already correctly extended — no masking or sign-fixing in the handler.

5.4 The read pointer — GETPTR and PB

Sometimes a handler needs to know *where* in hub memory the stream currently sits — to take a relative branch, or to compute a target address. **GETPTR** returns the FIFO's current hub pointer into D.

```
GETPTR ptr           ' stream address -> ptr
                    ' (or read PB under XBYTE)
```

XBYTE makes this automatic: on every dispatch it writes the FIFO pointer into **PB** (\$1F7), so a handler can read the current stream position from PB without issuing GETPTR itself. Between PA (the current bytecode, §6.3) and PB (the current stream pointer), a handler has both halves of its context handed to it automatically.

Chapter 6: LUT Dispatch

The last piece before the engine itself is the **dispatch table**: how a bytecode becomes a handler address. XBYTE keeps this table in LUT RAM, and its design is the reason dispatch costs only a few clocks.

6.1 The table is 256 EXECF operands in LUT

A cog's LUT is 512 longs. XBYTE uses a block of it — up to 256 longs — as the dispatch table, indexed by the bytecode. Bytecode *N* selects entry *N*. (Smaller tables are possible; Chapter 9 covers the size and compression options. The full-size case is 256 entries.)

Each entry is **one long**, and that long is an **EXECF operand** — exactly the operand Chapter 4 described:

- **bits [9:0]** — the handler's address in cog/LUT RAM
- **bits [31:10]** — the 22-bit SKIPF pattern applied on entry to the handler

The *Parallax Propeller 2 Documentation v35* states it directly: the table “*must consist of long data which EXECF would use, where the 10 LSBs are an address to jump to in cog/LUT RAM and the 22 MSBs are a SKIPF pattern to be applied.*”



Figure 6.1. LUT dispatch-table entry: one EXECF operand

CAUTION

Do not transpose the two fields. The address is the **low** 10 bits; the SKIP pattern is the **high** 22 bits. A handler that lives at LUT address \$200 with no skipping has the entry \$0000_0200, not the address shifted left. Build entries with the address in the low bits and the pattern shifted up by 10.

HARDWARE

The table is *this cog's* LUT — and 256 entries is the ceiling. A bytecode is one byte, so it indexes at most **256** entries, and the engine reads them from the LUT of the cog that is running it. You cannot enlarge the table by *sharing* LUT across two cogs: **SETLUTS** mirrors a companion cog's LUT *writes* into this cog's LUT — it does **not** merge the two into one address space, and each cog still has its own 512-long LUT. So when a guest needs **more than 256 distinct opcodes**, the answer is not more LUT but a **prefix bytecode that borrows an**

continues on next page →

alternate table with one-shot SETQ2 (§8.3, Chapter 17) — exactly how a CPU’s extended-opcode pages are handled. **Compression** (§9.3) is the complementary tool, for when many bytecodes *share* one handler rather than needing new ones.

6.2 Building a table entry

Because an entry packs an address and a pattern, build it by OR-ing the two fields. Assemblers let you compute this at assembly time:

```
' A dispatch entry: jump to 'push_const', no skipping.
      LONG      push_const          ' addr in [9:0],
                                     ' pattern [31:10] = 0

' A shared-body entry: jump to 'alu_body', skip pattern selects SUB.
      LONG      alu_body | (sub_skip << 10) ' addr | (pattern<<10)
```

The handler address comes from the label; the SKIP pattern is whatever leaves the instructions that bytecode needs and skips the rest (the shared-handler idiom of §4.4). The table is just 256 such longs, loaded into LUT before the engine starts.

6.3 The bytecode is handed to you in PA

When XBYTE dispatches bytecode *N*, it writes *N* into **PA** (\$1F6) before the handler runs. The handler can therefore use the bytecode value itself as data — as an immediate operand, a small constant, or an index — without re-reading it.

This matters for **compression** (Chapter 9), where a group of bytecodes shares one table entry and the handler tells them apart by reading PA. It is also simply convenient: a “push small constant” family can encode the constant *in the bytecode* and read it straight from PA:

```
MOV      value, pa          ' the bytecode is the datum
AND      value, #$0f        ' low nibble = constant 0..15
```

6.4 Dispatch, by hand

Putting Chapters 4–6 together, here is the dispatch loop XBYTE automates — written by hand, so the engine in Chapter 7 holds no surprises:

```
nextbc                                ' the hand-written loop
                                     ' fetch bytecode
      RFBYTE  index                    ' index into the LUT table
      ADD     index, #tbl_base
      RDLUT   entry, index             ' read the EXECF operand
      EXECF   entry                    ' jump + skip = dispatch
' ... each handler ends with a jmp back to #nextbc ...
```

Read a byte, use it to index the table, read the entry, execute it. That is fetch-look-up-execute — the loop from §3.1, in four instructions. **XBYTE is this loop in hardware**, with the return folded in so handlers end in `_RET_` and the engine re-enters the loop on its own — it also writes the bytecode to PA and the stream pointer to PB along the way, which the hand-written version above does not. Chapter 7 walks the hardware version clock by clock.

TIP

This loop is not a stepping stone. It is easy to read the next chapter and treat the hand-written version as scaffolding — needed once to understand the engine, never written again. It is the opposite: this loop is one you will come back to, for two reasons. It is:

- **your debug mode.** The engine's loop is hardware and has no body, so there is nowhere to put a `DEBUG()`. To trace which bytecode ran and where in the stream it came from, you take the engine out and run *this* instead (Chapter 11).
- **what most working P2 emulators actually ship.** The engine's auto-fetch requires the guest's code to live in hub — and a console's ROM does not — so they keep the EXECF dispatch and write the fetch themselves. That is this loop (Chapter 13).

The engine is a specialisation; this hand-written loop is the general case.

Part III: The XBYTE Engine

Part II built the pieces: the SKIP family, the FIFO stream, the LUT dispatch table. This part is the engine itself — the cycle that runs those pieces in hardware, the single instruction that arms it, the table-size and compression options that shape it, and the rules the handlers must follow. It is the reference for how XBYTE behaves.

Chapter 7: The Dispatch Cycle

XBYTE’s dispatch is the hand-written loop of §6.4, executed by hardware as a fixed sequence. The *Parallax Propeller 2 Documentation v35* specifies it as an **8-clock** sequence with a **6-clock overhead** per bytecode. This chapter walks it clock by clock — not because you write any of it, but because knowing exactly what the engine touches, and when, is what lets you reason about PA, PB, the flags, and timing inside a handler.

7.1 The eight clocks

Each bytecode dispatch runs this sequence. Clock 1 overlaps the `_RET_` that ended the previous handler, which is why the per-bytecode overhead is 6 clocks, not 8.

Clock	XBYTE activity	What it means
1	RFBYTE bytecode · SKIPF <code>#0</code>	fetch the next bytecode from the FIFO; cancel any leftover skip pattern from the previous bytecode
2	MOV PA, bytecode · RDLUT	write the bytecode to PA (\$1F6); begin the table read indexed by it
3	RDLUT (data → D)	the table’s EXECF operand arrives
4	EXECF D (begin)	start the dispatch jump-plus-skip
5	MOV PB, (GETPTR) · MODCZ · EXECF D (branch)	write the FIFO pointer to PB (\$1F7); set C,Z from the index if enabled; take the branch
6	flush pipeline	the branch flushes the pipeline
7	reload pipeline	the pipeline reloads at the handler
8	first handler instruction	the handler’s first instruction executes; on its closing <code>_RET_/RET</code> , loop to clock 1

The engine fetches, writes PA, looks up, writes PB, optionally sets flags, and branches — the entire fetch-look-up-execute loop — and hands control to your handler at clock 8.

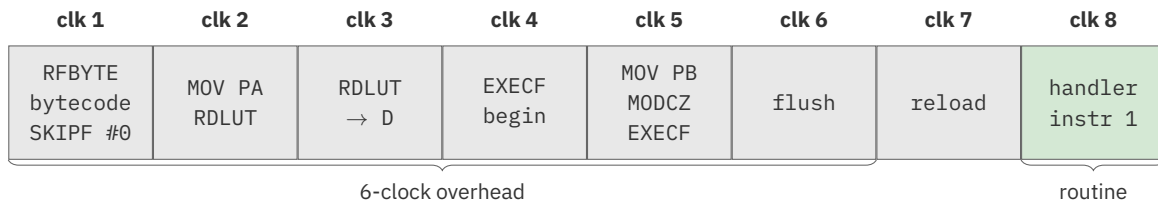


Figure 7.1. The 8-clock XBYTE dispatch cycle (clock 1 overlaps the prior `_RET_`)

7.2 The overhead, exactly

Three figures from the silicon documentation, stated precisely so they are not confused:

- **6 clocks** — the dispatch **overhead** per bytecode, “including `_RET_` at the end of each bytecode routine.” This is the tax XBYTE charges; the handler’s own work is on top of it.
- **9 clocks** — the overhead of the **software** equivalent (“2+3+4, or 9, clocks to get the next bytecode, look it up, then execute that bytecode’s routine”). XBYTE’s hardware saves the difference on every bytecode.
- **8 clocks** — the **minimum total loop**: a routine “as short as a single 2-clock instruction with a `_RET_` prefix” gives 6 overhead + 2 body.

HARDWARE

These are **hardware dispatch** clocks — a property of the engine, measured against the P2’s sysclk. They are not the run time of any particular interpreted language’s methods, which depend on the handlers a given interpreter ships. The figure to cite for XBYTE is the 6-clock overhead.

7.3 Flags

At clock 5 the engine can write **C** and **Z** from the low bits of the bytecode index, when the **F bit** is set in the mode operand (Chapter 9). When F is clear, dispatch leaves the flags alone, so a handler can carry flag state across bytecodes deliberately.

7.4 Interruption — and the fence you will need

XBYTE is **interruptible**. An interrupt can occur during dispatch, and the engine resumes the bytecode stream afterwards; bytecode interpretation does not lock out a cog’s interrupts.

That is good news, with a flip side worth stating plainly: **an interrupt can land in the middle of your handler.** The engine will resume the stream correctly afterwards — but it makes no promise whatever about the *work your handler was halfway through* when the interrupt fired. If that work had to be atomic, it has just been cut in half.

CAUTION

If a handler performs a multi-instruction sequence that must not be interrupted, you must fence it yourself.

The cases are easy to recognise once you know to look:

- a **CORDIC** operation — you issue the command, then collect the result some clocks later, and an interrupt in between will collect somebody else’s answer;
- a **read-modify-write** of a variable another cog can see;
- any sequence where a scratch register holds a half-finished value that a second entry to the same code would clobber.

The fence is **REP**. A REP block cannot be interrupted, so wrapping the critical sequence in a one-iteration REP shields it:

```

                REP    @.done, #1      ' shield: no interrupt inside
                QMUL   x, y            '   CORDIC command...
                GETQX  lo              '   ...and its result
.done
```

This is not a theoretical hazard, and it is not a rare one. The P2’s own Spin2 interpreter — the reference XBYTE program, written by the silicon’s designer — uses exactly this fence **repeatedly** (well over a dozen times in its source), guarding CORDIC operations and shared-variable updates.

The rule of thumb is simple: **XBYTE leaves interrupts working, and it is your job to say where they may not go.** A handler that only touches its own cog registers needs no fence at all. A handler that reaches for the CORDIC, or for memory another cog shares, needs one every time.

Chapter 8: Arming XBYTE

Starting the engine is a single instruction — but it is an unusual one, because it does two things at once and relies on a value already sitting on the hardware stack. This chapter covers the arming sequence, the \$1FF convention, and the persistent-vs-one-shot choice.

8.1 One instruction, with \$1FF on the stack

The *Parallax Propeller 2 Documentation v35* puts it plainly: “Starting XBYTE and establishing its operating mode is done all at once by a `_RET_ SETQ {#}D` instruction, with the top of the hardware stack holding \$1FF.”

So arming XBYTE takes two things in place:

1. **\$1FF on top of the hardware stack** — this is the address every bytecode routine “returns to.” When a handler ends in `RET/_RET_`, the return target \$1FF is what triggers the engine to fetch and dispatch the next bytecode.
2. **A `_RET_ SETQ {#}D`** — the `_RET_` prefix performs the return (consuming the \$1FF) and, in the same step, **SETQ** loads the **mode operand D** that configures the engine.

<code>RDFAST #0, ##program</code>	1. point the FIFO at the bytecode stream
<code>PUSH #\$1ff</code>	2. return target for every bytecode routine
<code>_ret_ SETQ #\$100</code>	3. arm: 256-entry EXECF table at LUT \$100 — XBYTE starts now

After that `_ret_ SETQ`, the engine is running: it fetches the first bytecode and dispatches it, and keeps going until stopped. The \$1FF is **not consumed** as it goes — the *Parallax Propeller 2 Documentation v35* is explicit that the triggering return “does not pop the stack,” so the single `PUSH` you did at arm time serves every dispatch that follows, sitting on the hardware stack for as long as the engine runs. (That it is never popped has a consequence when you eventually leave the engine — §10.4.)

CAUTION

The \$1FF must be on the stack before the arming `_RET_`, and you put it there with `PUSH #$1FF`.

Without it, the `_RET_` simply returns wherever the stack happens to point and **the engine never engages** — silently. Nothing faults; your program just runs on as if you had never armed it, so the cause can be slow to find.

continues on next page →

In particular, **a CALL will not do the job for you.** A CALL pushes *its own* return address, which is not \$1FF. Every working implementation — the documentation’s own demo included — writes the explicit PUSH.

8.2 The mode operand

The D value handed to SETQ is the **mode operand**. It packs three independent choices, all detailed in Chapter 9:

- the **table base address** in LUT (the high bits),
- the **table size / compression** selection (which bit pattern), and
- the **F bit** (bit 0) — whether dispatch writes the flags.

For a full 256-entry table at LUT base \$100 with flags untouched, the operand is \$100 — table base in the high bits, the size/F bits clear. Chapter 9 is the full map.

HARDWARE

Bit 1 of the mode operand selects the index form.

Across the smaller-table modes (§9.2), bit 1 chooses *which half* of the bytecode indexes the dispatch table: 0 indexes from the bytecode’s **low** bits, 1 from its **high** bits — the latter freeing the low bits as an operand in PA. The §9.2 patterns show it directly: %AAxx0010F and %AAxx0011F are the *same* 128-entry mode, differing only in bit 1.

In the **256-entry mode** the bytecode already fills all eight index bits, so there is no low/high choice to make and **bit 1 is simply ignored** — a genuine don’t-care in that mode.

What to do about it: leave it 0 unless you specifically want a smaller table’s high-bits index form. Every arming idiom in this book leaves it 0, which selects the low-bits form (and in 256 mode makes no difference either way).

8.3 Persistent vs one-shot — SETQ and SETQ2

There are two arming instructions, and the choice between them is **orthogonal** to the operand value. The operand says *how* to dispatch; SETQ-vs-SETQ2 says *for how long*:

- **SETQ** arms the **persistent** mode. The configuration is retained and applies to **every** subsequent bytecode. This is how you start the engine and how it stays running.
- **SETQ2** arms a **one-shot** mode that applies to **exactly the next bytecode**, after which the engine automatically reverts to the mode last set by SETQ — “*without having to restore the original XBYTE mode afterwards.*”

The one-shot form lets a VM keep a default dispatch table and *borrow* an alternate table for a single bytecode at a time. The classic use is a **prefix bytecode**: a bytecode whose handler issues `_ret_ SETQ2` to select an alternate table, so the *following* bytecode dispatches through that alternate table and then control returns to the default. This is exactly how a guest CPU’s “extended opcode” pages are handled — the subject of the 6809 vignette in Chapter 17.

	SETQ	SETQ2
Persistence	persistent — every bytecode	one-shot — the next bytecode only
After it fires	stays in effect	reverts to the last SETQ mode
Typical use	arm and run the engine	a prefix/alternate-table bytecode

TIP

The “2” in SETQ2 is the alternate/one-shot form throughout the instruction set — the same personality split as the block-move SETQ/SETQ2. If you remember “SETQ2 = the temporary one,” you will reach for the right one when building a prefix bytecode.

CAUTION

SETQ2 does two entirely different jobs, and only the *next instruction* tells them apart.

You have already used it both ways in this book, and it is worth stopping to notice:

```

                SETQ2  #256-1           ' (a) BLOCK MOVE:
                RDLONG $100, ##table    '      load 256 longs into LUT

_ret_  SETQ2  #alt_mode                ' (b) ONE-SHOT XBYTE MODE:
                                     '      next bytecode only

```

Same mnemonic. In **(a)** it is a block-transfer count, consumed by the RDLONG that follows. In **(b)** it is an XBYTE mode operand, consumed by the `_RET_`. Nothing about the SETQ2 itself distinguishes them — **the instruction that follows decides what it meant.**

The two never collide in practice, because a block move is followed by a memory instruction and an arming is followed by a return. But a SETQ2 read out of context tells you nothing, and a misplaced one fails in a way that will not look like the mistake you made. When you read someone else’s XBYTE code — or your own, six months on — **look at the next line first.**

Chapter 9: Table-Size & Compression Modes

Modes

A bytecode set rarely needs all 256 codes, and LUT RAM is shared with everything else a cog does. XBYTE therefore supports smaller dispatch tables and a compression scheme, all selected by the **mode operand** handed to SETQ/SETQ2. This chapter is the reference for those modes and for the **F bit**.

9.1 Reading the mode operand

The mode operand is written %A...F — a high field **A** that sets the LUT base address, a middle pattern that selects the table size, and the low **F** bit for flag writing. The same value chooses table base, size/compression, and flags at once. The table below is the full set from the *Parallax Propeller 2 Documentation v35*.



Figure 9.1. The mode operand %A000000xF (256-entry table form)

9.2 Table sizes

Every table size except 256 comes in **two index forms**, selected by **bit 1** of the operand (§8.2): the **primary** form (bit 1 = 0) indexes from the bytecode's *low* bits; the **alternate** form (bit 1 = 1) indexes from its *high* bits, leaving the low bits free as an operand in PA. Here is the full set — every form the silicon accepts:

LUT size	Index bits	Operand pattern	LUT base	Index from bytecode
256	8	%A000000xF	%A00000000	I = b[7:0]
256 + compression	8	%ABBBB00xF (BBBB > 0)	%A00000000	b[7:4] < BBBB → b[7:0]; else group (§9.3)
128 primary	7	%AAxx0010F	%AA0000000	I = b[6:0]
128 alternate	7	%AAxx0011F	%AA0000000	I = b[7:1]
64 primary	6	%AAAx1010F	%AAA000000	I = b[5:0]
64 alternate	6	%AAAx1011F	%AAA000000	I = b[7:2]
32 primary	5	%AAAAx100F	%AAAA00000	I = b[4:0]
32 alternate	5	%AAAAx101F	%AAAA00000	I = b[7:3]
16 primary	4	%AAAAA110F	%AAAAA0000	I = b[3:0]
16 alternate	4	%AAAAA111F	%AAAAA0000	I = b[7:4]

The more **A** bits the operand carries, the higher the LUT base can sit and the smaller the table — a 16-entry table needs only 4 index bits, so the other bits position it. The **256-entry mode is the exception to the bit-1 rule**: the bytecode already fills all eight index bits, so there is no low/high choice to make and bit 1 is ignored there (§8.2). In its place, the 256 mode offers **compression** (§9.3) rather than an alternate index form.

9.3 Compression — 16 primary plus 240 extended

The 256-entry mode has a compression option, written %ABBBB00xF with the 4-bit threshold **BBBB** greater than zero. It lets “sets of 16 bytecodes, which would use identical LUT values, ... be represented by a single LUT value, effectively compressing blocks of 16 LUT values into single LUT values.”

The rule is on the bytecode's high nibble:

- if **b[7:4] < BBBB**, the bytecode indexes the table normally (I = b[7:0]) — these are the **primary** bytecodes, each with its own entry;
- if **b[7:4] ≥ BBBB**, the whole group of 16 that shares that high nibble maps to a **single** entry — these are the **extended** bytecodes.

This is most useful “*when the bytecode, which is always written to PA, is used as an operand within the bytecode routine.*” A family like “push constant 0..15” can be 16 bytecodes that share one handler — the handler reads the actual value from PA. Sixteen bytecodes, one table entry, one routine.

9.4 The F bit — flags from the bytecode

Bit 0 of the mode operand is the **F bit**:

- **F = 0** — dispatch does not affect C or Z; flag state carries across bytecodes.
- **F = 1** — at clock 5, dispatch writes **C = bytecode index bit 1** and **Z = bytecode index bit 0**.

Setting F lets a routine “*differentiate behavior within a bytecode routine, especially in cases of conditional looping, where a SKIPF pattern would have been insufficient, on its own.*” In other words, when one handler must behave four slightly different ways and a SKIP pattern cannot express the difference, encode two selector bits in the bytecode’s low bits and let the flags carry them in:

```
' Armed with F=1: each dispatch has set Z = index bit 0, C = index bit 1.
' One shared handler reads those two bits straight from the flags:
    if_z    JMP    #odd_variant      ' bytecode's low bit was 1
    if_c    ADD    step, #1          ' next bit selects a behaviour
```

HARDWARE

The F bit and the SKIPF pattern are complementary selectors. The SKIP pattern chooses *which instructions* a handler runs; the flags (via F) let those instructions *branch* on two bits of the bytecode. Compression, the SKIPF pattern, and the F bit together are how a small table serves a large, regular bytecode set.

9.5 A real mode operand, decoded

Everything in this chapter is easier to trust once you have taken a real one apart. So here is the mode operand the **P2's own Spin2 interpreter** arms with — the reference XBYTE program, written by the silicon's designer:

```
_ret_   SETQ   #1A1          ' Spin2 interpreter's operand
```

\$1A1 is nine bits: %1_1010_0001. Lay it against the compression pattern %ABBBB00xF from §9.3:

Field	Bits	Value	Meaning
A	1	%1	table base = %A000000000 = LUT \$100
BBBB	4	%1010	compression threshold = \$A
00	2	%00	the 256-entry-with-compression selector
x (bit 1)	1	%0	index-form select — ignored in 256 mode (§8.2), so 0 here
F	1	%1	flags written from the bytecode index

Read it back out in words: *a 256-entry dispatch table at LUT \$100; bytecodes \$00–\$9F get individual entries; bytecodes \$A0–\$FF compress — each group of sixteen sharing one entry and one handler, which reads the actual bytecode from PA; and dispatch writes C and Z from the bytecode's low bits.*

This single operand exercises §9.2, §9.3 and §9.4 at once.

TIP

Work the other way when you design your own: decide the **base** (where in LUT can you afford 256 longs?), decide the **threshold** (how many bytecodes genuinely need their own handler, and where does the regular, operand-carrying tail begin?), then decide the **F bit** (do any handlers want two selector bits in the flags?). Concatenate, and you have your operand. The three choices are independent — that is the point of packing them into one value.

Chapter 10: Bytecode Routines

The handlers are the code you write. XBYTE runs the loop between them, but the routines themselves must follow a few rules so the engine can re-enter cleanly. This chapter collects those rules and the idioms that come from them.

10.1 The rules a routine follows

Rule	Why
Live in cog or LUT RAM	dispatch is an EXECF jump to a 10-bit cog/LUT address; hub-resident routines cannot be the EXECF target
End in RET or _RET_	the return to \$1FF is what tells the engine to fetch the next bytecode
Respect the hardware stack (8 levels)	nested CALLs inside a handler share the cog's 8-level stack; overflowing it corrupts returns

Within those rules a handler is ordinary PASM2. It can call subroutines, read and write hub, drive pins — anything a cog can do — as long as it returns to \$1FF when done.

CAUTION

Watch the stack depth. XBYTE's re-entry rides on the hardware stack, and the stack is only 8 levels deep. A handler that calls a subroutine that calls another is fine; one that recurses deeply, or leaves calls unbalanced, will exhaust the stack and break dispatch. Keep handler CALL chains shallow.

10.2 The bytecode as an operand — PA

Because the engine writes the bytecode to **PA** (\$1F6) before the handler runs, the handler can use the bytecode itself as data. This is what makes compression (§9.3) and the small-constant idiom work:

```
' "push small constant 0..15" — 16 bytecodes, one entry, one routine.
' The value lives in the low nibble of the bytecode, read from PA.
push_small
        MOV     value, pa
        AND     value, #$0f           ' the constant is in PA[3:0]
        _ret_   CALL    #push_value   ' onto the VM stack, return
```

10.3 Inline operands – the FIFO and PB

A handler that needs a larger operand pulls it from the **FIFO**, which is sitting on the byte right after the bytecode (§5.3). Reading it with RFVAR/RFVARS/RFLONG advances the stream so the next dispatch lands correctly:

```
' "jump relative" – a signed offset follows the bytecode in the stream.
jmp_rel
        RFVARS  offset          ' signed inline operand
        GETPTR  ptr             ' or read PB for current pos
        ADD     ptr, offset
        _ret_   RDFAST #0, ptr   ' re-point the FIFO -> branch
```

Re-pointing the FIFO with RDFAST is how a guest “branch” works under XBYTE: change where the stream reads from, and the next bytecode comes from the new location. PB gives the handler the current position to compute the target from.

HARDWARE

PB is also how you get the stream back after you have left it.

Sooner or later a handler must do something the FIFO cannot survive — call out to hub code for a device, run a routine that itself uses RDFAST, hand work to another cog and wait. Any of those may leave the FIFO pointing somewhere else entirely, and when your handler returns, the engine will happily fetch the next “bytecode” from wherever the FIFO now happens to be.

The fix is two instructions, and it is the standard idiom:

```
' Handler that calls out to hub code, then resumes the stream cleanly.
op_port_write
        RFBYTE  port            ' the port number, inline
        CALL    #hub_write_port ' ...may disturb the FIFO
        ADD     pb, #1           ' step past the operand
        _ret_   RDFAST #0, pb    ' re-point the FIFO and carry on
```

PB held the read position from the moment the engine dispatched this bytecode (clock 5, §7.1), so it is still a valid anchor even after the FIFO has been used for something else. Adjust it for whatever operand bytes you consumed, RDFAST back to it, and the next dispatch lands exactly where it should.

10.4 Stopping the engine

XBYTE runs until a handler chooses **not** to return to \$1FF. A “halt” bytecode’s handler simply does not end in the dispatch-continuing return — it branches to ordinary code instead, leaving the engine. That is the clean way to exit: one bytecode whose handler jumps out of the loop rather than back into it.

CAUTION

If the cog will re-arm, reclaim the \$1FF first. The arming \$1FF was never popped (§8.1) — so the moment a handler jumps *out* of the loop instead of returning to it, that \$1FF is still sitting on the hardware stack. For a **run-once** VM that halts and parks (the shape of §12.2), that is harmless; the cog never touches the stack again. But a **reusable interpreter cog** — one that finishes a job, drops back to an idle wait, and later arms *again* for the next job — must **POP** that stale \$1FF as it exits, or the hardware stack gains one entry per job. With only eight levels, and any handler or between-jobs code that also uses CALL/RET, that drift eventually starves the stack — and it **wraps with no fault** to warn you. The rule is simple: if you will arm more than once, POP on the way out.

```
h_halt      POP      tmp      ' reclaim the arming $1FF
            JMP      #idle    ' back to the between-jobs wait
```

(Appendix C’s minimal community example, §C.8, is a reusable cog that does exactly this.)

Chapter 11: Debugging XBYTE

You have now written handlers, built a table, and armed the engine. Sooner or later it will not do what you meant, and you will want to look inside it — which is where XBYTE presents its one genuinely awkward property.

The engine's loop is hardware. There is no loop body. In a software interpreter you would drop a `DEBUG()` into the dispatch loop and watch every instruction go by. XBYTE has no such place: it goes from your handler's `_RET_` to the next handler's first instruction in six clocks, with nothing of yours in between. (Chapter 13 shows how far the consequences of that reach; this chapter is about living with it.)

Fortunately, the silicon anticipated this.

11.1 What the hardware will tell you

GETBRK reads the engine's live state. It requires a flag effect, and the flag you choose selects *which* information you get.

GETBRK D WC — the engine's configuration:

Field	Meaning
C	the LSB of the active skip pattern
D[31:28]	CALL depth since the pattern began — skipping is suspended when this is not zero
D[27]	1 = plain SKIP · 0 = SKIPF / EXECF / XBYTE
D[26]	LUT sharing enabled
D[25]	XBYTE pending on the next <code>_RET_/RET</code>
D[24:16]	the 9-bit XBYTE mode operand — the very value you armed with
D[15:0]	event-trap flags

GETBRK D WZ — the pattern itself:

Field	Meaning
Z	set when no skip pattern is queued (and D = 0)
D	the full 32-bit SKIP/SKIPF/EXECF/XBYTE pattern, used LSB-first

Between them, these answer *what the engine is doing right now*: whether it is armed, in which mode, and what pattern is currently queued. Note what is **not** among them — how many instructions remain in the current bytecode routine. The queued pattern bits are visible, but a routine ends at its `_RET_`, which is a property of your code, not a length the engine tracks.

HARDWARE

Notice `D[31:28]` — the `CALL` depth — and the sentence attached to it: **skipping is suspended while it is non-zero**. This is not a debugging curiosity; it is a load-bearing fact about the engine, and it is the reason a handler can `CALL` a shared helper at all. The helper's instructions are **not** eaten by the caller's `SKIP` pattern, because the pattern is suspended for the duration of the call. Chapter 15's `_RET_ CALL #set_nz` idiom depends entirely on this.

11.2 The debugger shows you the engine's state

You rarely need to call `GETBRK` yourself, because the P2's single-step debugger does it for you. Its display carries **the live 32-bit skip pattern** and **the 9-bit XBYTE mode**, side by side with the registers and the program counter.

Better still: in the disassembly view, **an instruction that the current skip pattern will cancel is drawn struck through**. You can see the pattern working — which instructions of a shared body are live for this bytecode and which have been skipped away. For a `SKIPF` pattern that is off by one bit, this view is the quickest way to spot the error.

And XBYTE survives being debugged, for a reason worth knowing: the debug interrupt enters and exits through its dedicated debug-interrupt vectors (`IJMP0/IRET0`, via `RETI0`) rather than the 8-level `CALL` stack. Because it never touches that stack, the `$1FF` the engine depends on (§8.1) is left undisturbed across every breakpoint. You can stop the world, look around, and let it run on. (A debugger that itself makes calls still has to budget the 8-level stack — the ROM does not manage it for you.)

11.3 The technique the engine cannot give you

The debugger steps **P2 instructions**. That is exactly right when your handler is misbehaving — and exactly wrong when your question is *“which bytecode ran, and where in the stream was it?”* No amount of stepping through hardware dispatch will show you a **guest-level** trace, because the dispatch is not made of instructions you can stop on.

The answer is a direct one: **take the engine out and put the loop back**.

Recall §6.4, where you dispatched by hand before meeting XBYTE. That was not merely a teaching device. It is the **debug mode** — a software loop that does exactly what the engine does, with one crucial difference: **it has a body you can write in**.

Comment out the two arming instructions, and substitute this:

```
' Debug mode: the software equivalent of the engine, with a place to stand.
' Trades ~3 clocks per bytecode for a guest-level trace.
landing      NOP                                ' pad: absorbs a leftover
```

continues on next page →

↪ continued from previous page

```

NOP                                     ' skip pattern from the
NOP                                     ' previous handler
NOP
NOP
NOP
NOP
NOP
NOP
dispatch  RFBYTE  pa                     ' fetch the bytecode  (clock 1)
           GETPTR  pb                     ' stream position    (clock 5)
           DEBUG(uhex_byte(pa), uhex_long(pb))
           RDLUT   entry, pa              ' table entry      (clock 2-3)
           PUSH    #landing               ' handlers _RET_  back to here
           EXECF   entry                  ' jump + skip     (clock 4-5)

```

Every line maps onto a clock of the hardware cycle (Chapter 7) — that is the point. The engine’s behaviour is reproduced exactly, and now PA and PB pass through code you own, so a `DEBUG()` can print **which bytecode** and **where in the stream** on every single dispatch. That is a guest-level trace, and the hardware cannot give it to you.

CAUTION

The landing pad is not decoration — it prevents a specific trap.

A SKIP pattern is consumed as instructions execute. If a handler’s pattern is *longer than the handler* — more bits than there are instructions to cancel — the leftover bits do not evaporate. They fall on **whatever executes next**.

Under XBYTE this is harmless: the engine issues a `SKIPF #0` at clock 1 of every dispatch (Appendix A), cancelling any leftover pattern before the next bytecode runs. **Your software loop does no such thing.** The leftovers land on the first instructions of *your loop*.

And you cannot simply cancel it at the top of the loop, because **a leftover pattern would cancel your cancel** — a skipped `SKIPF #0` never executes, and so never clears anything. Hence the pad: enough NOPS to absorb the longest pattern you emit, harmlessly, before the real loop begins.

The root cause is worth fixing at the source: **do not emit a pattern longer than the handler it belongs to.** Size each pattern to its body and the problem never arises — in which case the pad costs you nothing and insures you anyway.

11.4 Leaving the switch in

Both modes are only a couple of instructions, so build your program to hold them both from the start:

```

' Arm the engine - the fast path.
           PUSH    #$1FF                     ' XBYTE fires on RET to $1FF
_ret_     SETQ    #mode                     ' ...and away it goes

```

continues on next page →

↔ continued from previous page

' Debug path: comment out the two above; use the software loop.

The cost of keeping the software loop in your source is a few longs of cog space. The cost of *not* keeping it is rediscovering, at the worst possible moment, that your hardware dispatch loop has no window in it.

TIP

Two failure modes, two tools. If a **handler** is wrong — the wrong variant ran, the flags came out strange — use the debugger and read the SKIP pattern; the strikethrough will usually show you the bug directly. If the **stream** is wrong — the wrong bytecode ran, or you branched somewhere unintended — take the engine out and trace the loop.

Part IV: Building Interpreters and Emulators

Parts II and III explained the engine — and Chapter 11 showed you how to see it running. This part proves it by building. Chapter 12 builds a complete, working bytecode VM from nothing: the smallest thing that exercises the whole engine. Chapter 13 then steps back and asks the question that comes *before* any emulator — which of the engine’s assets you can actually take, and what each one costs — and Chapter 14 answers it for the classic guest processors, one by one. Chapters 15 through 17 build a tiny 6502, service its interrupts, and handle prefix bytes with alternate tables. Chapter 18 closes the part by widening the frame off interpreters entirely: the same engine parsing protocols, decoding formats, and driving displays.

Everything in this part is **tiny and illustrative** — sized to show a technique end to end, not to be a faithful or complete implementation. (The two complete programs, the Chapter 12 VM and the Chapter 18 display list, compile; the shorter handlers elsewhere are representative fragments.) That is a deliberate charter.

Chapter 12: A Minimal Custom VM

The smallest useful XBYTE program is a stack machine with a handful of bytecodes. Building it touches every part of the engine once: load a dispatch table into LUT, point the FIFO at a bytecode program, arm the engine, and write a few handlers. This chapter is that build, complete.

12.1 The instruction set

Four bytecodes are enough to be a real VM:

Bytecode	Name	Action
\$00	PUSHC	read a variable-length constant from the stream, push it
\$01	ADD	pop two, push their sum
\$02	SUB	pop two, push their difference
\$03	HALT	leave the engine

The VM stack is a few longs of cog memory with a pointer. PUSHC uses an inline operand (§5.3); ADD and SUB are a shared body candidate (§4.4), but are written separately here for clarity.

12.2 The complete program

```

CON
    _clkfreq = 200_000_000

DAT
    ORG

' ---- start the VM -----
start
    SETQ2    #4-1                ' load 4 longs into LUT $000..
    RDLONG   $000, ##disp_table  ' the dispatch table
    RDFAST   #0, ##program       ' FIFO -> bytecode stream
    PUSH     #$1ff               ' return target per bytecode
    _ret_    SETQ    #0           ' arm: 256-entry table @ LUT
                                ' $000, F=0 - XBYTE starts now
' XBYTE now runs the stream; control reaches 'done' only via HALT.
done
    JMP      #done               ' park the cog

' ---- bytecode handlers (cog-resident) -----
h_pushc
                                ' $00: push inline constant
    RFVAR    v
    WRLONG   v, sp
    _ret_    ADD     sp, #4

h_add
                                ' $01: pop b, a; push a+b
    SUB      sp, #4
    RDLONG   b, sp
    SUB      sp, #4
    RDLONG   a, sp
    ADD      a, b
    WRLONG   a, sp
    _ret_    ADD     sp, #4

h_sub
                                ' $02: pop b, a; push a-b
    SUB      sp, #4
    RDLONG   b, sp
    SUB      sp, #4
    RDLONG   a, sp
    SUB      a, b
    WRLONG   a, sp
    _ret_    ADD     sp, #4

h_halt
    JMP      #done               ' $03: exit XBYTE

' ---- cog variables -----
sp      LONG    stack           ' VM stack pointer (hub)
v        RES     1
a        RES     1
b        RES     1

' ---- hub data: dispatch table, program, stack -----

```

continues on next page →

↪ continued from previous page

```

    ORGH
disp_table  LONG    h_pushc      ' entry $00 -> PUSHC
             LONG    h_add       ' entry $01 -> ADD
             LONG    h_sub       ' entry $02 -> SUB
             LONG    h_halt      ' entry $03 -> HALT

program     BYTE    $00, 7      ' PUSHC 7
             BYTE    $00, 5      ' PUSHC 5
             BYTE    $01         ' ADD      -> 12
             BYTE    $00, 3      ' PUSHC 3
             BYTE    $02         ' SUB      -> 9
             BYTE    $03         ' HALT

stack       LONG    0[16]      ' VM stack lives in hub

```

That is a working XBYTE VM. The dispatch table is four longs in hub, loaded into LUT \$000 with SETQ2+RDLONG; each entry is a handler’s cog address with a zero SKIPF pattern, so no SKIPF is needed here. The program is a byte stream in hub; RDBFAST points the FIFO at it. The `_ret_ SETQ #0` arms the simplest mode — a 256-entry table at LUT base \$000, flags off; only the first four entries are populated because the program uses only bytecodes \$00–\$03. The engine runs until HALT’s handler jumps to done.

HARDWARE

Why the table entries are just handler addresses. With no skipping, an entry’s SKIPF pattern (bits [31:10]) is zero, so the long is simply the handler’s cog address in bits [9:0]. The moment you adopt the shared-handler idiom (§4.4), those high bits fill with per-bytecode SKIP patterns — see §12.3.

12.3 Folding ADD and SUB into a shared body

ADD and SUB differ by one instruction. Per §4.4 they can share a body, with each bytecode’s table entry carrying the SKIP pattern that selects its operation:

```

alu                                     ' shared ADD/SUB body
    SUB      sp, #4
    RDLONG   b, sp
    SUB      sp, #4
    RDLONG   a, sp
    ADD      a, b                      ' kept by ADD, skipped by SUB
    SUB      a, b                      ' kept by SUB, skipped by ADD
    WRLONG   a, sp
_ret_ ADD     sp, #4

```

ADD’s entry skips the `SUB a, b` line; SUB’s entry skips the `ADD a, b` line. The table changes from two handler addresses to two `alu`-with-pattern entries; the body exists once. For two bytecodes the

saving is small, but for a dozen ALU operations it is the difference between one routine and a dozen.

12.4 In practice: the engine usually runs in its own cog

The VM above arms XBYTE in the same cog that set it up, then parks — ideal for seeing the whole thing at once. A real interpreter is almost always different in one structural way: **the engine gets a cog to itself**. The Spin2 interpreter does this, and so does the community XBYTE VM in §C.8.

The shape is always the same. A launching cog builds an image whose LUT already holds the dispatch table, starts a cog on it with **COGINIT**, and shares a small **mailbox** in hub — a few longs the two cogs use to say “*here is the program*,” “*here is where to put the result*,” and “*go*.” The interpreter cog’s *first act* is the arming sequence of Chapter 8; from then on the exit rules of §10.4 — including the re-arm POP if it will serve more than one job — are what keep it re-usable.

This guide stops at the engine and leaves cog orchestration (COGINIT, mailbox protocols) to the general P2 references. The pattern is named here only because it is where most projects go the moment their first bytcodes run. For a complete, minimal, community-written instance of exactly this shape — a dedicated XBYTE cog fed by a hub mailbox, popping its \$1FF between jobs — see **Appendix C, §C.8**.

Chapter 13: The Three Decisions

Everything so far has taught the engine. This chapter is the judgement that comes *before* you use it: which of the engine’s assets your guest can actually take. The intuitive test is not the one that decides it.

The intuitive test is to ask: *does my guest’s instruction shape fit the engine?* Byte-stream and opcode-first, and you are in the sweet spot; word opcodes or fixed-width words, and you are not. That test is tidy, but it does not predict what real emulators do. There are working P2 emulators for guests that are *perfectly* byte-stream and opcode-first, and they do not use XBYTE at all. There are others whose instruction shape is a poor fit, and they use half of it very happily.

Instruction shape is not the axis. **Three decisions are**, and this chapter is about them.

13.1 Two assets, three decisions

XBYTE gives you two separable things — this much of the classic story is exactly right:

1. **Auto-fetch** — the FIFO pulls the next byte and the engine dispatches it with **no software in the loop**.
2. **Table/EXECF dispatch** — one indexed jump-plus-skip selects the handler.

They are genuinely independent: you can take the second without the first. But to see *when* you can take each one, you have to notice that an interpreter is really three mechanisms, not two:

Decision	The question	What answers it
The fetch	Where do the guest’s <i>instructions</i> come from?	the FIFO — or code you write
The dispatch	How do you get from opcode to handler?	a jump table · EXECF · XBYTE
The memory model	How does the guest read and write its <i>data</i> ?	hub · external RAM · memory-mapped I/O

The dispatch is **independent of the other two**, and one emulator lets you see it directly. Marco Maccaferri’s P2 8086 PC-XT emulator was published in two variants that differ only in where the guest’s memory lives: one keeps it in hub, the other in external PSRAM. Diff the two and **about a hundred lines change out of more than eight thousand — and not one of them is in the dispatch**. The opcode table, the decode, the handler jump: untouched. Swap the entire memory backend and the dispatch never notices. (Both variants, and a second 8086 emulator that takes a different dispatch rung, are in Appendix C.)

That independence is the result the rest of the chapter builds on.

13.2 The dispatch ladder

Dispatch is not a yes-or-no question about XBYTE. It is a **ladder with three rungs**, and you may stop on any of them.

Rung	What you write	What the P2 does for you
1 — jump table	fetch the opcode · index a table · JMP through it	nothing special — this works on any processor
2 — EXECF dispatch	fetch the opcode · index a table · EXECF through it	the skip pattern rides along, so one handler body serves many opcodes (§4.4)
3 — XBYTE	<i>nothing</i> — arm it once	fetch and dispatch, in hardware, for six clocks

The rungs are not merely hypothetical. The **same guest processor** — the 8086 — has been emulated on the P2 at rung 1 *and* at rung 2, by different hands (Appendix C). One reads its opcode, indexes a table in hub, and jumps:

```
' rung 1 - a plain jump table
    CALL    #read_opcode    ' fetch it yourself
    PUSH    #next_op       ' your own return address
    JMP     opimpl         ' ...and a plain jump
```

The other builds EXECF entries and lets the SKIP pattern collapse whole families of opcodes onto shared bodies:

```
' rung 2 - EXECF dispatch, skip pattern along for the ride
    CALL    #read_opcode    ' still fetch it yourself
    PUSH    #next_op       ' still your own return address
    EXECF   opimpl         ' ...but now jump AND skip
```

Look at what is the same and what changed. Both hand-roll the **fetch**. Both PUSH their **own** return address. The only difference is the last instruction — and that difference buys the whole shared-body idiom.

TIP

A key point: **you can take the dispatch asset without taking the engine.** EXECF plus a LUT table is available to any program, any time, with no arming, no \$1FF, and no constraint on where the guest's code lives. Most working P2 emulators live exactly here, on rung 2; §13.3 explains why.

13.3 The coupling decision

They stop at rung 2 because of the fetch, and the fetch is not a feature choice but a coupling decision — one you make on the first day.

The FIFO reads **hub memory**. That is the whole of it, and everything follows:

XBYTE's auto-fetch requires the guest's code to live in hub RAM.

If the guest's program fits in hub, auto-fetch is free speed and you should take it. A bytecode language, a small stack machine, an 8-bit micro's ROM — these fit, and the engine carries them beautifully.

But a console's ROM is *megabytes*. It lives in external PSRAM or HyperRAM, and the FIFO cannot reach it. Not awkwardly — **at all**. So the emulator must supply its own fetch: a routine that pulls bytes from external memory, usually through a prefetch queue. And the moment you write that routine, XBYTE's auto-fetch has nothing left to do.

Now recall §13.1's measurement — the emulator published in a hub variant and a PSRAM variant, a hundred lines apart, dispatch untouched. That was possible because they had hand-rolled the fetch: their fetch went through the memory path like every other access, so when the memory backend changed, the fetch followed with it.

Had they taken auto-fetch, that port would have forced a far larger rewrite — auto-fetch welds the guest's code to hub, and undoing that means replacing the fetch throughout (§14.9 notes the one narrow exception).

CAUTION

Auto-fetch is fast, and it welds your guest's code to hub RAM. A hand-rolled fetch costs you clocks and buys you a **swappable memory backend**.

If there is any chance your guest will outgrow hub, do not take the auto-fetch — undoing it later means a rewrite of the fetch path.

And there is a second, quieter cost. XBYTE reads its dispatch table from **LUT**, and LUT is a cog resource that the rest of your system also wants — for a prefetch queue, a palette, a line buffer, a sine table. An emulator that needs LUT for one of those must move its dispatch table off LUT, and XBYTE goes with it. The FIFO is contended the same way: a cog streaming a video framebuffer through the FIFO cannot also use it to fetch instructions, even if the guest's code lives in hub. Chapter 3's resource budget (§3.6) is not a formality — it is the second half of this decision.

Where the guest — and its assets — come from. All of that is about where the guest's code *runs*; a separate question is where it *comes from*. A guest's program, its ROMs, and its media are loaded into hub or PSRAM before anything executes, and on a P2 Edge module they load from one of two places on the module itself: the **16 MB SPI flash** or the **microSD socket** — which, on the Edge modules, **share the same four pins (P58–P61)**, so you reach one or the other, not both at once.

Flash suits a single fixed guest that boots on power-up; a microSD card is the natural home for the bulky, swappable things — a shelf of ROMs, a library of disk images — read on demand through a community SD/FAT driver. Either way the shape is *load, then run*: nothing is fetched from the card at instruction speed.

Which is why you do not page the guest from storage. When a guest outgrows hub, the answer is to hold its image **whole in PSRAM** — the 32 MB module (**P2-EC32MB**) maps all 32 MB as one linear space, room for any classic guest and its RAM several times over. Swapping pieces of code in and out from microSD as the guest runs — an overlay or demand-paging scheme — is the wrong shape for this machine: a card read is orders of magnitude slower than a PSRAM access, and a guest branches wherever it likes, so there are no quiet boundaries to confine the swapping to. Load the image into PSRAM once, and let the card go back to sleep.

And PSRAM is a shared road. The 32 MB module reaches its PSRAM over a single external bus, and that one bus carries everything external: the emulator’s guest-memory traffic *and*, on most projects, the **framebuffer the display streams to show the guest’s screen**. Capacity is not the contest — 32 MB holds a large guest and a framebuffer with room to spare; **bandwidth is**, and the display’s share is continuous and unforgiving (starve the framebuffer and the picture tears). So you budget the bus: size the display mode so its stream leaves headroom for the emulation, or keep a small guest in hub and give PSRAM wholly to the picture, or lean on a PSRAM driver that arbitrates between cogs — MegaYume’s does exactly this — so a video cog and an emulation cog can share the road without the picture breaking.

13.4 There is no loop body

The third decision is the subtlest, and it is why some emulators decline XBYTE for reasons that have nothing to do with memory.

XBYTE’s loop is hardware. There is no loop body.

Consider what that means. In a software interpreter, the dispatch loop is a *place* — a few instructions that run once per guest instruction, where cross-cutting work naturally lives. Real emulators put a great deal there. The dispatch loop of a real Z80 core is a busy place; between one guest instruction and the next it may:

- **arbitrate the bus** it shares with another guest processor;
- reserve a **hook slot** — a NOP that can be patched into a jump — for breakpoints and tracing;
- **pace the emulation**: compute elapsed time against the guest’s cycle budget and WAITX to throttle the P2 *down* to real Z80 speed;
- set up per-instruction register state;
- clear the **prefix state** left by the previous instruction;
- tick the Z80’s **refresh register**, which the guest’s own software can read.

That is a great deal of cross-cutting work, in one place, once per instruction. (MegaYume’s Z80 core does the bus-arbitration, pacing, and refresh among them — §C.4.)

Under XBYTE, **none of that has anywhere to live**. The engine goes from your handler's `_RET_` straight to the next handler's first instruction, in hardware, in six clocks. There is no gap. Every one of those six concerns would have to be replicated inside *every handler* — or dropped.

HARDWARE

This is the trade, stated plainly:

XBYTE gives you hardware dispatch and takes away the one place where per-instruction work naturally lives.

A software loop costs you roughly three extra clocks per instruction and gives you **a place to stand**. Whether that is a bargain or a disaster depends entirely on how much cross-cutting work your guest demands — and cycle-accurate emulation of real hardware demands a great deal.

You can see the consequence most clearly in debugging. An emulator that *does* use XBYTE, when its author needed to trace guest execution, had no choice: **comment the engine out** and substitute the software dispatch loop of §6.4, with a `DEBUG()` in the middle. There was nowhere else to put it. An emulator that never armed XBYTE simply leaves a `NOP` in its loop and patches it when needed. Same problem; one of them pays nothing. Chapter 11 makes this practical.

13.5 Choosing, in order

The three decisions have a natural order, because each one constrains the next.

1. **Where will the guest's code live?** If it fits in hub, auto-fetch is available. If it must live in external memory, auto-fetch is off the table — **and so is XBYTE**, because the engine is fetch-and-dispatch together. Stop at rung 2.
2. **How much per-instruction cross-cutting work does the guest demand?** Cycle-accurate timing, interrupt polling, bus sharing, refresh registers, tracing. If the answer is “a lot,” you want a loop body, and XBYTE takes it away. Stop at rung 2.
3. **Is LUT free?** XBYTE needs 256 longs of it. If your palette, prefetch queue, or line buffer has already claimed LUT, the table goes to hub — and XBYTE cannot read a table in hub. Stop at rung 2.

Three roads to rung 2, and only one combination — **code in hub, little cross-cutting work, LUT free** — arrives at rung 3.

That is a precise result, and it maps exactly onto what XBYTE was built for: **an interpreted language**. A bytecode VM keeps its program in hub, does no cycle-accurate anything, and wants its LUT for exactly one thing. It is no coincidence that the P2's own Spin2 interpreter is the engine's showcase — it is the shape XBYTE was designed around, and the shape it serves best.

Chapter 14 turns these three decisions into a per-processor survey: what each classic guest will actually cost you.

13.6 Why the 6502

The capstone in Chapter 15 is the **6502**, and this chapter's framework makes the choice concrete:

- **Its code fits in hub.** A 6502's entire address space is 64 KB — it fits in hub with room to spare, so auto-fetch is genuinely available. This is decision one, and the 6502 passes it where a console does not.
- **Byte-stream, opcode-first.** Every instruction begins with a one-byte opcode followed by 0–2 operand bytes. RFBYTE fetches the opcode; RFBYTE/RWORD pull its fixed-width operand bytes — **not** RFVAR/RFVARS, which decode variable-length (self-sizing) values and would mis-read any operand byte $\geq \$80$. (RFVAR belongs to a bytecode VM's operands — §12.2 — not a fixed-width guest CPU.)
- **A table that fits.** The 6502 defines about 151 of 256 opcodes — a 256-entry table maps them directly, one bytecode per opcode.
- **Regular families.** Its addressing modes and ALU operations are regular enough that the shared-handler idiom (§4.4) collapses many opcodes onto a few bodies — a natural showcase for SKIP patterns.

The **8080**, **Z80**, and **8051** fit equally well and would make fine alternates. The 6502 is chosen for familiarity.

CAUTION

Be clear about what the capstone is *not*. Our 6502 is a **teaching artifact** — it takes rung 3 because rung 3 is what this book is about. A 6502 emulator that had to be **cycle-accurate**, or whose ROM lived in external memory, would land on rung 2 like everything else in Appendix C. The technique is what transfers; the rung is a decision you make for *your* guest, not one this book makes for you.

Chapter 14: What Will Hurt

A Guest-CPU Survey

Chapter 13 gave you three decisions. This chapter answers them for the processors people actually emulate, so you can see at a glance what you are signing up for.

Two tables, because a reader arrives with two different questions. The first is “*can I use the engine at all?*” The second is “*what is going to hurt me regardless?*” They are not the same question, so they get separate tables.

14.1 How to read this survey

A guest processor is not hard or easy in the abstract. It is hard or easy **relative to the three decisions**, and the survey is organised that way.

One honesty marker, which matters:

A • in the **Real?** column means a working P2 implementation of this guest exists, and the row reports **what it actually does**. Appendix C will point you at it.

An unmarked row applies Chapter 13’s model to the guest’s *documented* behaviour. That model has been checked against every marked row in this table — but a row **without** the mark is **reasoning, not observation**, and you should hold it a little more loosely than a marked one. So should we.

14.2 Can you take the engine?

The first decision dominates: **where does the guest's code live?** The FIFO reads hub, so a guest whose program fits in hub can be auto-fetched, and one whose program cannot, cannot.

Guest	Real?	Guest address space	Instruction shape	Realistic rung
6502 / 65C02		64 KB — fits hub	byte, opcode-first	3 — XBYTE
8080	•	64 KB — fits hub	byte, opcode-first	3 — XBYTE
Z80	•	64 KB — fits hub	byte, opcode-first	2 — needs cycle pacing
6809		64 KB — fits hub	byte, opcode-first	3 — XBYTE
8051		64 KB code — fits hub	byte, opcode-first	3 — XBYTE
CHIP-8		4 KB — fits hub	2-byte, nibble-decoded	3 — via compression (§9.3)
65816	•	16 MB — off-chip	byte, opcode-first	2 — the ROM cannot be streamed
68000	•	16 MB — off-chip	16-bit word opcodes	2
x86 (8086)	•	1 MB, segmented	byte, but CS:IP	2
ARM / MIPS		—	32-bit fixed words	2 , or JIT

CAUTION

The 65816 row is the one to dwell on. It is byte-stream and opcode-first — by instruction shape *identical* to the 6502, which sits comfortably at rung 3. And the working P2 implementation of it uses **neither** XBYTE nor auto-fetch, because a 65816 machine's ROM is megabytes and lives off-chip.

The instruction shape did not decide it; the address space did — the lesson of Chapter 13 in a single row.

14.3 What will hurt anyway?

The rung you land on says nothing about how hard the *guest* is. These costs are yours no matter which rung you stand on.

Guest	Prefixes	Flags	Decimal	Guest interrupts	Cycle accuracy
6502 / 65C02	none	N,V,Z,C — cheap	D flag on ADC/SBC	IRQ + NMI vectors	needed for games
65816	none	+ M/X width bits	D flag	IRQ, NMI, ABORT	required
8080	none	+ auxiliary carry	DAA	RST vectors	modest
Z80	CB/ED map · DD/FD modifier	full F, incl. H and N	DAA	modes 0/1/2 + NMI	required
6809	\$10/\$11 — map	CC register	DAA	IRQ, FIRQ, NMI	modest
8051	none	PSW	DA A	5 sources, 2 levels	modest
68000	none	CCR + the X bit	ABCD/SBCD	7 levels, vectored	required
x86 (8086)	\$0F map (286+) · segment/REP/LOCK modifier	the lazy-flags problem	AAA/DAA family	INT + the IF flag	modest
ARM / MIPS	—	NZCV (ARM)	none	exception vectors	rarely
CHIP-8	none	VF only	none	none — timers only	timers only

The rest of this chapter takes each column in turn, because *what the column costs you on the P2* is the part a survey table cannot say.

14.4 Prefixes are two different things

A distinction that is easy to get wrong: two kinds of prefix that look identical in a hex dump and want opposite treatment. A **map prefix** — 6809 \$10/\$11, Z80 CB/ED, x86 \$0F (286+) — selects a *different opcode map*, changing *which* handler runs; one-shot SETQ2 hands you the alternate table. A **modifier prefix** — x86 segment/REP/LOCK, Z80 DD/FD — changes *how* a handler behaves, not which one runs, and wants a **state register**, not a table. Confuse the two and you either build a table you never needed or decode the wrong instruction — and the **Z80 carries both**, which makes it the sharpest example. Chapter 17 builds both mechanisms in full; at survey level the cost is simply *knowing which kind each of your guest's prefixes is*.

14.5 Flags — the cost nobody budgets for

The P2 has two flags: *c* and *z*. Your guest almost certainly has more, and every one of them must be *computed*, *stored*, and *read back* — on every instruction that touches them.

For many guests this is the largest hidden cost, and it scales with how faithful you must be.

- **6502** is kind: *n* and *z* fall out of the result almost free, and *c/v* are cheap. A single shared `set_nz` helper serves most of the instruction set.
- **Z80 and 8080** are not kind. The Z80's *F* register carries **H (half-carry)** and **N (subtract)** bits that exist for one reason: to make DAA work afterwards. They are pure bookkeeping — no program reads them directly — and you must maintain them anyway, on every arithmetic instruction, or DAA produces the wrong answer.
- **x86** is where emulators traditionally cheat, and the cheat has a name: **lazy flags**. Rather than compute all six status flags on every ALU operation, you store the operands and the operation, and only *derive* the flags if something actually reads them. In practice many instructions' flags are never read. It is a large win and a large complication.

TIP

The *F* bit (§9.4) can help here, but not as a guest flag register. It does not carry your *guest's* flags — it writes the **bytecode's own low bits** into *c* and *z* at dispatch. That is a way to let one handler body branch four ways on which opcode selected it; it is not a guest flag register. Your guest's flags live in a cog register you maintain yourself.

14.6 Decimal mode

Decimal mode is easy to under-test: almost every 8-bit guest has one, and a test program rarely exercises it until real software trips over it.

The 6502's *D* flag silently changes what *ADC* and *SBC* *mean*. The 8080, Z80 and 6809 instead provide *DAA*, which corrects a binary result to a packed-BCD one **after the fact** — and to do that, *DAA* must know the half-carry and (on the Z80) whether the last operation was an add or a subtract. That is why those bookkeeping flags in §14.5 exist, and why you cannot SKIP them.

The honest advice: **decimal mode is small, fiddly, well-specified, and testable**. Write it early, test it against a known-good table, and never think about it again. Deferred, it tends to resurface later as a puzzling bug — a miscomputed score, a wrong address — that is slow to trace back to decimal mode.

14.7 Guest interrupts

Every guest but CHIP-8 has them, and servicing them under a *hardware* dispatch loop is the problem Chapter 16 exists to solve. Two things are worth knowing here, at survey level:

- **The guest's interrupt-enable flag is just a cog register you own.** DI and EI become one instruction each.
- **Where you poll matters more than how.** With a software loop you poll once, in the loop. Under XBYTE **there is no loop** (§13.4) — so the poll must live inside handlers, at points where interrupting is safe. That is a design decision, not a detail, and it is the clearest practical consequence of taking rung 3.
- **The guest decides *when* it accepts, not just whether.** Enable-delays (the Z80/8080 EI waits one instruction), prefix and atomic sequences that hold interrupts off, and interruptible-and-resumable instructions are all part of the guest's architecture — a faithfulness dial most emulators leave off, and one §16.3 shows how to honour when a guest's own code depends on it.

The Z80's three interrupt modes and the 68000's seven vectored levels are more *bookkeeping* than the 6502's single IRQ line, but the mechanism is the same in all of them.

14.8 Cycle accuracy

Ask this question early, because the answer changes your architecture.

If the guest drives real hardware whose timing is visible — a video signal, an audio channel, a raster interrupt — then instruction-level timing is not enough. You must count the guest's cycles and *pace* the emulation to them. Real implementations do this by computing elapsed time against the guest's cycle budget and using WAITX to throttle the P2 **down** to the guest's speed, once per instruction.

And now the catch: **that per-instruction pacing has nowhere to live under XBYTE** (§13.4). This is why cycle accuracy and rung 3 pull against each other, and why the Z80 row in §14.2 carries the caveat it does.

If your guest is a language runtime, a scripting VM, or a self-contained program with no externally visible timing, you need none of this — and rung 3 is yours.

14.9 Edge cases

The survey tables stop where the honest advice becomes "*it depends on your guest.*" Three things routinely bite, and none of them are P2-specific:

Undocumented opcodes. The 6502's illegal opcodes and the Z80's IX/IY half-register instructions are used by real software — demos and copy-protection especially. They cost you table entries, not architecture. Decide up front whether you are emulating the *specification* or the *silicon*; they are different machines.

Self-modifying guest code. Harmless when your handlers read the guest's memory on every fetch — which, if you hand-rolled the fetch, they do. But if you have taken **auto-fetch**, the FIFO may have already read ahead past code the guest just rewrote. Another quiet consequence of

rung 3, and one more reason a self-modifying guest wants rung 2 — though not an absolute one. You *can* keep auto-fetch and still honour a rewrite by re-priming the FIFO with RDFAST before the affected fetch: RDFAST re-initialises the FIFO, discarding the bytes it prefetched past the change, so the next read comes fresh from hub. The catch is that re-priming makes the FIFO refill from hub every time — erasing the very prefetch that made auto-fetch worth taking — so doing it on every instruction pays rung 2's price while still on rung 3. (A guest that rewrites code only occasionally can be smarter: flush *only* when a store lands in the code region, not blindly per instruction.) The technique is real; it is almost never the right trade, and the usual answer stays rung 2.

Memory-mapped I/O and banking. The guest's address space is rarely flat. A clean way through: make the memory-access routine a **register holding an address**, so the routine itself can be swapped per region — ROM, RAM, I/O, banked window. CALL through it, and the map becomes data instead of a chain of comparisons.

14.10 Reading the survey for your own guest

Nothing in these tables is magic, and the method transfers to a guest that is not listed. Ask, in this order:

1. **Does the guest's code fit in hub?** No → rung 2. Stop.
2. **Does anything about it need cycle-accurate pacing?** Yes → rung 2, most likely. Stop.
3. **Is LUT free?** No → rung 2. Stop.
4. **Is the first byte an opcode that can index a table?** Yes → rung 3 is genuinely available to you.

Then, whichever rung you land on, the columns of §14.3 are your work list — flags, decimal, prefixes, interrupts — and those you owe your guest regardless of what the P2 does for you.

Chapter 15: A Tiny CPU Emulator (6502)

This chapter builds an illustrative slice of a 6502 on XBYTE — enough opcodes to show how a real CPU's instructions become bytecode handlers, and how the engine's pieces (PA, the FIFO, the shared body, the SKIP pattern) carry the emulation. It is **not** a complete or cycle-accurate 6502, by charter; it is the technique, shown end to end on a representative handful of instructions.

15.1 The mapping

In a 6502 emulator the correspondence is direct:

6502 concept	XBYTE realization
the opcode byte	the bytecode — fetched by the engine, handed to the handler in PA
operand bytes (immediate, address)	inline operands — RFBYTE/RWORD off the FIFO (fixed-width)
the opcode → microcode decode	the dispatch table — one entry per opcode
the program counter	the FIFO read position — advanced by reads, re-pointed by RDFAST on a branch
A, X, Y, S, P registers	cog registers

The guest's program counter *is* the FIFO position: incrementing the PC is automatic (the FIFO advances as it reads), and a branch is a RDFAST to the new address.

15.2 Register file and dispatch

The 6502 register set is a few cog longs, and arming is the same sequence as Chapter 12 — a 256-entry table for the full opcode space:

```

SETQ2    ##256-1          ' load 256 longs into LUT
RDLONG   $100, ##op_table ' the opcode dispatch table,
                        ' LUT $100..$1FF
RDFAST   #0, ##reset_vector ' FIFO -> 6502 code in hub
PUSH     #$1ff
_ret_    SETQ    #$100      ' 256-entry table @ LUT $100,
                        ' F=0 (see §8.2)
```

Each opcode that the slice implements gets a table entry pointing at its handler; unimplemented opcodes point at a shared `op_undef` that flags the gap. (A real build fills all 256; this slice fills the few below and routes the rest to `op_undef`.)

15.3 Representative handlers

LDA #imm (\$A9) — load the accumulator with an immediate. The operand byte follows the opcode in the stream; N and Z flags are set from the result.

```
op_lda_imm                                ' $A9: LDA #immediate
      RFBYTE  a                            ' inline operand -> A
      _ret_   CALL  #set_nz                ' update N,Z; return to dispatch
```

INX (\$E8) — increment X, a single-byte instruction with no operand, flags from the result. The mask keeps the guest register 8-bit.

```
op_inx                                    ' $E8: INX
      ADD     x, #1
      AND     x, #$ff
      _ret_   CALL  #set_nz                ' 6502 X is 8-bit
                                           ' update N,Z; return to dispatch
```

JMP ABS (\$4C) — an absolute jump: read the 16-bit target from the stream, then re-point the FIFO at it. This is where “the PC is the FIFO position” pays off.

```
op_jump_abs                              ' $4C: JMP $hhll
      RFWORD  target
      _ret_   RDFAST #0, target            ' 16-bit absolute address
                                           ' FIFO -> target = the branch
```

The shared-body idiom collapses the 6502’s many ALU and load/store opcodes the way §12.3 collapsed ADD/SUB: one body per family (loads, ALU ops, branches), with each opcode’s table entry supplying the SKIPF pattern and, where four-way behavior is needed, the F bit (§9.4) carrying two opcode bits into the flags.

15.4 What this slice shows, and what it omits

The slice demonstrates the full technique: opcode-as-bytecode, operands from the FIFO, PC-as-FIFO-position, branches as RFAST, and shared bodies for regular families. A faithful 6502 adds the rest of the opcode table, the full addressing-mode matrix, decimal mode (§14.6), correct flag semantics on every operation (§14.5), interrupt servicing (Chapter 16), and accurate timing (§14.8) — none of which change the XBYTE technique, all of which are deliberately out of scope here. The point is the shape of the solution, not a finished emulator.

One omission would shape a real build, so it gets its own note.

TIP**The addressing-mode matrix wants a second dispatch, not a bigger table.**

The 6502 has 56 operations and 13 addressing modes. The brute-force route gives each combination its own table entry — hundreds of nearly identical handlers.

Working emulators do something better — the 68000 core in Appendix C among them: **dispatch twice**. The opcode's table entry carries not only its handler but a small field naming its **addressing mode**. A first, shared block — indexed by that field — computes the effective address (read the operand, apply the index register, handle page wrap, add the cycle penalty). *Then* the opcode's own handler runs, and finds its operand already waiting.

Opcode → addressing mode → operation. One shared addressing block instead of a mode's worth of duplication in every handler.

It is the industrial-strength version of the shared-body idiom in §4.4, and it is what keeps a full instruction set manageable.

Chapter 16: Servicing Guest Interrupts

Your guest has an interrupt line. Almost all of them do — and Chapter 15’s 6502 is no exception, with its IRQ and NMI vectors sitting at the top of memory waiting to be honoured.

In a software interpreter this is a solved problem so ordinary that nobody writes it down: you check the interrupt line once per pass, at the top of the dispatch loop, and if one is pending you push the guest’s program counter and vector to its handler. One check, one place.

XBYTE has no dispatch loop. The engine goes from `_RET_` to the next handler in hardware, and there is nowhere to put the check. This chapter is how real emulators solve that.

16.1 The guest’s interrupt state is just cog registers

Start with the easy half. Everything the guest knows about its own interrupts — the enable flag, the pending state, the mode — is **yours to keep in cog registers**. There is no P2 mechanism involved and no cleverness required.

The guest’s interrupt-enable flag is one bit that you own, so the guest’s DI and EI instructions become one instruction each:

```
op_di  _ret_  BITL    inte, #0          ' guest DI - disable interrupts
op_ei  _ret_  BITH    inte, #0          ' guest EI - enable them
```

That is the whole of it. A guest instruction that manipulates the guest’s interrupt state is just a handler that manipulates your register.

16.2 Getting the signal in

The interrupt itself comes from *outside* your emulation cog — a timer cog, a video cog reaching a raster line, an I/O cog with a byte to deliver. So you need a way for another cog to say “*something happened*” that costs you nothing while nothing is happening.

The P2 has exactly that, in the **attention** mechanism. Another cog raises it; your cog tests it with **JATN**, a jump-if-attention that costs two clocks and never blocks:

```
JATN    #int_pending    ' anything waiting? (2 clocks)
```

That is a two-clock, non-blocking poll — exactly what this needs.

16.3 Where to poll — the real question

Now the hard half, and it is a **design decision**, not a technique.

Under a software loop, you poll once, at the top, and every guest instruction is an interrupt boundary. Simple, uniform, correct.

Under XBYTE there is no top. The poll has to live **inside handlers** — and you must choose *which* handlers, because putting it in all of them costs two clocks on every guest instruction and puts you back where a software loop would have left you.

The choice has a real consequence:

Poll in...	Interrupt latency	Cost
every handler	one guest instruction — ideal	2 clocks × every instruction
a shared body that many opcodes route through	bounded by that body's reach	2 clocks, paid once per family
only control-flow handlers (branches, calls, returns)	until the guest next branches	nearly free

A shipped 8080 emulator takes the third road: it polls in the shared body that ends its **control-flow** instructions, on the reasoning that a guest's program counter is unambiguous at a branch and that real 8080 code branches constantly. That is a defensible engineering trade, not a universal answer — a guest running a long unrolled loop would see its interrupts arrive late.

CAUTION

Choose the safe points deliberately. An interrupt must only be taken where the guest's state is *consistent* — the previous instruction fully retired, no half-computed address in a scratch register. Handlers that have already finished their work and are about to return are safe; the middle of a multi-step addressing-mode computation is not.

This is the sharpest practical consequence of taking rung 3 (§13.4). The engine gave you hardware dispatch and took away the place where the check naturally belonged, so **you** now decide where interrupt boundaries live. Decide it once, write it down, and be consistent.

“Consistent state” is only half the rule — the guest's architecture fixes the other half. *Where* you may safely inject is a property of *your* handlers; *when the guest will accept* an interrupt is a property of the *guest*, and the two are not the same. Real processors defer, block, or allow interrupts at points their own designers chose, and a faithful emulator honours those rules rather than taking an interrupt wherever a poll happens to fall:

- **Enable-delay.** On the Z80 and 8080, EI does not take effect until *after the next instruction* — so EI followed by RET cannot be interrupted between the two, and interrupt-return code

depends on that to finish cleanly. (The 6502 has its own one-instruction quirk around CLI /SEI.) Model it by making the enable *pending*: a flag that the *following* instruction promotes to “enabled,” not the EI handler itself. Poll where you like — but gate *acceptance* on the delayed flag.

- **Atomic sequences that block acceptance.** The Z80 holds interrupts off *across a prefix* — a DD/FD/CB/ED byte and the opcode it modifies are indivisible, and no interrupt may land between them. If your prefix handling spans two dispatches (Chapter 17), the poll must not fire on the first.
- **Interruptible-and-resumable instructions.** The opposite case: the Z80’s block moves (LDIR) and the x86 string repeats (REP MOVSB) can be interrupted *partway* and *resumed* — the guest keeps its progress in its own registers and re-enters where it left off. Implement such an instruction as a P2 loop and the interrupt boundary lives *inside* it, at each iteration, not only at its end.

None of this is mandatory. Most P2 emulators poll at convenient points and never reproduce the enable-delay, because the guest software they run does not lean on it. It matters only when the guest’s own code does — an interrupt-return that assumes one more instruction runs, a driver that toggles the enable in a tight sequence. Treat it as a **faithfulness dial**, set by what your guest actually needs, exactly like decimal mode (§14.6) or cycle accuracy (§14.8) — but know the dial is here, because the “consistent state” rule above will otherwise let an interrupt through a full instruction too early.

16.4 Injecting the interrupt

The mechanism rests on one observation about the dispatch table.

A dispatch-table entry is **just a long** — a handler address in the low ten bits, a SKIP pattern above (§6.1). The engine reads one from LUT and hands it to EXECF. But nothing says an EXECF operand has to *come* from the table. **You can build one yourself and execute it.**

So servicing a guest interrupt is not a special mechanism at all. It is a **bytecode that never came from the stream**:

```
int_pending
    TESTB    inte, #0          wc  ' are guest interrupts enabled?
    if_nc    JMP      #int_ignore  ' no - resume the stream
            BITL     inte, #0      ' yes - guest clears IE
            MOV     guest_pc, pb   ' remember where the guest was
    _ret_    EXECF   int_vector_entry  ' ...and "dispatch" it
```

`int_vector_entry` is a table entry you constructed — the address of your interrupt-sequence handler, OR’d with whatever SKIP pattern that handler needs:

```
int_vector_entry LONG int_go | (%0 << 10) ' a hand-built dispatch entry
```

The engine does not know, and does not care, that this bytecode was not in the stream. It jumps and skips exactly as it would for a real one.

TIP

This generalises. **EXECF is the dispatch primitive; the table is merely the usual place to keep its operands.** A whole class of problems opens up once you see it that way — synthesising a dispatch, chaining one handler into another, or building an entry from parts at run time. The interrupt is simply the first place you need it.

16.5 Halt, and waiting for an interrupt

Most guests have an instruction that stops the processor until an interrupt arrives — the 8080's HLT, the 6502's various idle idioms. It looks like it needs special support, but it does not — the FIFO handles it.

JNATN is the mirror of JATN: jump if there is **no** attention. So a halt handler spins on it. And when no interrupt has arrived, the handler must arrange for the guest to **execute the same instruction again** — which, because the guest's program counter is the FIFO position (§10.3), means backing the stream up by one byte:

op_halt	JNATN	#.still_halted	' no interrupt yet?
	JMP	#int_pending	' one arrived - go service it
.still_halted	SUB	pb, #1	' back the stream up one byte...
ret	RDFAST	#0, pb	' ...and re-run this same HLT

Four instructions: the handler re-executes HLT until an attention arrives, then hands off to `int_pending` — which services the interrupt if the guest has them enabled, or resumes the stream if it does not. (A guest that must halt *forever* with interrupts disabled — the true 8080 semantics — would gate that exit on the enable flag; this illustrative version wakes on any attention.)

16.6 What this costs you

Nothing here is expensive in clocks. JATN is two, the injection is one EXECF, and the guest's interrupt-enable flag is a single bit you were keeping anyway.

What it costs is **a decision you would not have had to make** on a software loop: *where are my interrupt boundaries?* The engine bought you three clocks per bytecode and handed you that question in exchange. For a language interpreter — which has no interrupts to service — it costs

nothing. For a CPU emulator it is a real, if modest, tax, and one more entry on the ledger of Chapter 13.

Chapter 17: Prefixes and Alternate Tables

Almost every guest processor eventually runs out of opcodes and solves it with a **prefix byte** — a byte that changes the meaning of the byte after it. XBYTE has an instruction that looks made for exactly this: the one-shot **SETQ2** (§8.3), which borrows an alternate dispatch table for precisely one bytecode and then reverts on its own.

SETQ2 is *an* answer to prefixes — but only to half of them. Knowing which half is what separates a clean prefix implementation from a tangle.

17.1 Prefixes are two different things

Look at what a prefix actually *does* to the byte that follows it, and they fall into two groups that want opposite treatment:

	Map prefix (escape)	Modifier prefix
Examples	6809 \$10/\$11 · Z80 CB/ED · x86 \$0F (286+)	x86 segment override, REP, LOCK · Z80 DD/FD
What it changes	which handler runs — a <i>different opcode map</i>	how the handler behaves — same instruction, different memory or register
The tool	one-shot SETQ2 — an alternate table	a state register , then re-fetch

A **map** prefix genuinely redirects dispatch. After the 6809's \$10, the byte \$83 names a different instruction than \$83 alone does. You want a different table, and one-shot SETQ2 hands you one — and reverts on its own.

A **modifier** prefix does *not* want dispatch redirected. An x86 segment override does not change *which* instruction runs — it changes *which memory* that instruction touches. Pointing it at an alternate table would mean duplicating every handler once per segment register, which is absurd. The right answer is a register (§17.3).

CAUTION

The Z80 carries both kinds, which makes it the best teacher and a genuine trap. Its CB and ED prefixes are **map** prefixes — different opcode tables. Its DD and FD prefixes are **modifier** prefixes — they retarget HL to IX or IY and leave the opcode map alone.

Treat DD like CB and you will build a duplicate table you never needed. Treat CB like DD and you will decode the wrong instruction entirely.

17.2 Map prefixes – the 6809’s pages

The Motorola 6809 is a clean small demonstration, so it carries the worked example.

It extends its opcode space with two **prefix bytes**, \$10 and \$11. An opcode introduced by \$10 belongs to “page 2”; one introduced by \$11 belongs to “page 3”. The same second byte means different things depending on which prefix — if any — preceded it. A flat 256-entry table cannot express that: byte \$83 is one instruction on page 1 and another on page 2.

This is exactly what one-shot SETQ2 is for. Keep the page-1 table as the persistent mode (armed once with SETQ). Make \$10 and \$11 *prefix bytecodes* whose handlers do nothing but select the matching alternate table **for the next bytecode only**:

```

op_page2
    _ret_    SETQ2    #page2_mode    ' $10: select page-2 table
                                     ' next bytecode dispatches
                                     ' through page 2, then reverts

op_page3
    _ret_    SETQ2    #page3_mode    ' $11: select page-3 table
                                     ' next bytecode dispatches
                                     ' through page 3, then reverts

```

When the 6809 stream contains \$10 \$83, the \$10 handler arms the page-2 table one-shot; the \$83 that follows dispatches through page 2; and the engine then reverts to page 1 on its own — “*without having to restore the original XBYTE mode afterwards.*” No flag to clear, no mode to put back. The prefix byte costs one dispatch, and the alternate page is in effect for exactly the one opcode that needs it.

What matters here is what is **absent**. Written by hand, a prefix means keeping an “am I in a prefix?” flag and testing it before every single opcode. Here there is no flag and no test: the prefix shifts the machine for exactly one dispatch, and normal decoding resumes with no code to put it back.

TIP

This is the general pattern for **any map prefix** — the 6809’s pages, the Z80’s CB/ED, the x86 \$0F escape. Each becomes a one-shot-SETQ2 handler pointing at the alternate table for that page, and SETQ2’s automatic revert does the bookkeeping for you.

17.3 Modifier prefixes — state, not dispatch

Now the other half, which SETQ2 cannot help with.

An x86 segment-override prefix says “*the next instruction’s memory access goes through this segment.*” It does not change which handler runs. So its handler simply records the fact and goes back for the real opcode:

```
' A modifier prefix: remember it, then fetch the instruction it modifies.
op_seg_es      MOV      override, seg_es      ' remember which segment...
               JMP      #next_op              ' ...and fetch the opcode
```

Every memory-touching handler then consults `override`, and clears it once the instruction retires. That is the whole mechanism. The Z80's `DD/FD` are the same idea wearing a different hat: they do not select a table, they change which register `HL` means for one instruction.

HARDWARE

Be honest about what this costs under `XBYTE`.

Setting state and returning is fine — the engine dispatches the next bytecode, and the modified handler consults the register. No problem.

The awkward case is a modifier that must **re-execute** the following instruction rather than merely colour it. x86's `REP` is the notorious example: it runs the *next* instruction over and over until a counter expires. `XBYTE` fetches a **new** bytecode on every `_RET_`, so a `REP` handler cannot express “run that one again” as a plain return. It must fetch and dispatch the repeated instruction **itself**, in a loop — which is to say it hand-rolls exactly the loop the engine was supposed to save it (§6.4).

The engine still dispatches everything else; it simply has nothing to offer this one pattern, which is worth knowing before you design around it.

17.4 SETQ2 is bigger than prefixes

One-shot `SETQ2` is more than “the prefix trick,” and the `Spin2` interpreter shows why.

The `P2`'s own `Spin2` interpreter runs **two dispatch tables at once** — its main bytecode table, and a second table of *variable operators* at a different LUT base. Handlers that need the second one end like this:

```
_ret_  SETQ2  #var_op_mode      ' the next bytecode is a
                                ' variable operator
```

The second table does not hold an *extended page* of the same kind of thing — it holds a *different kind of thing*, a variable operator decoded through its own table. The bytecode stream has a small *grammar* — some bytecodes are operations, others are operands-with-behaviour that must be decoded through their own table — and one-shot `SETQ2` is what expresses it.

Prefixes are merely the most obvious instance of a much larger idea:

The table *is* state. Change the table and you change what the machine *is* — for one bytecode with `SETQ2`, or for good with `SETQ`.

Seen that way, a whole class of designs opens up: a two-stage bytecode grammar, a parser whose states *are* tables, a decoder that switches interpretation mid-stream. Chapter 18 takes that idea well outside emulation, into some of XBYTE's less obvious uses.

Chapter 18: XBYTE Beyond Interpreters

Parts II–IV taught XBYTE as the engine under a language or a CPU, because that is what it was built for and where it shines. But strip the word “bytecode” away and the machine is more general: **XBYTE is hardware table-driven dispatch over a byte stream**. It reads the next byte, indexes a table, jumps to a handler, and loops — and nothing in that sentence requires the stream to be a *program*. Any problem shaped like *walk a stream of bytes, and for each one do one of a small set of things* can ride the same six-clock loop. This chapter widens the lens: the engine that runs a VM will also parse a protocol, decode a format, drive a display, or sequence a show.

18.1 Two assets, three widening features

Chapter 13 named XBYTE’s **two separable assets** for a CPU emulator; they are just as useful outside emulation:

- **Auto-fetch** — the FIFO pulls the next byte and dispatch happens with no software in the loop. Any *ordered byte stream* gets this.
- **Table/EXECF dispatch** — one indexed jump-plus-skip selects the handler. Any *first-byte selector* gets this.

An application draws on whatever mix of the two its data calls for. Three features you have already met turn that mix into a wide range of uses:

Feature	Taught in	What it unlocks
The byte is data — it lands in PA, and compression lets a group share one handler	§6.3, §9.3, §10.2	the symbol doubles as an operand or index: a channel number, a small constant, a packed length
The table is state — SETQ2 borrows an alternate table for one byte; SETQ swaps it for good	§8.3	escape and prefix bytes, mode shifts, whole state machines: change the table, change what the machine <i>is</i>
The stream can seek — PB gives the position, RDFSFAST re-points it	§5.4, §10.3	loops, jumps, replays, back-references: the read cursor is a free, movable “program counter”

The rest of the chapter applies these to problems that are not interpreters — with the same honest fit-grading Chapter 13 used for CPUs, because the engine helps some of them far more than others.

18.2 The application map

A survey of where XBYTE earns its keep beyond languages and CPUs. Fit is graded **strong** (both assets plus a widening feature), **partial** (leans on one asset), or **marginal** (the dispatch is really just a lookup).

Application	The stream is...	Main lever	Fit
Terminal / ANSI (VT100) reader	characters + escape sequences	table-as-state (ESC → one-shot table)	strong
MIDI stream engine	status + data bytes	byte-as-data (channel in PA)	strong
Graphics display list	draw commands + inline coordinates	seek + auto-fetch	strong
Binary format / TLV decoder (MessagePack, CBOR)	type-tagged records	byte-as-data (the type tag)	strong
Event / timeline sequencer (LED art, tracker, animatronics)	time-ordered events	seek (loop / branch)	strong
Stream decompression (multi-code RLE, packed sprites)	control tokens + payload	auto-fetch — dispatch pays only with <i>many</i> codes	partial
Inter-cog command coprocessor	a live command ring	dispatch — the stream is live, not stored	partial
Lexer / protocol state machine	input symbols	table-as-state, one table per DFA state (advanced)	partial
Forth inner interpreter	a threaded word stream	both — but interpreter-adjacent	partial
Charset map / Morse / template expand	symbols → output	marginal — a plain RDLUT wins unless there is real per-symbol work	

The three sketches that follow take one **strong** case for each widening feature. Each is **tiny and illustrative** — a handful of handlers to show the shape, not a finished driver — the same charter as the rest of Part IV.

18.3 A terminal reader — the table as state

A serial terminal receives a stream of output bytes. Most are printable characters to place on screen; a few are controls, and one — \$1B, ESC — announces that *the next byte begins an escape sequence* that must be read from a different set of rules. That is exactly one-shot SETQ2 (§8.3): the ESC handler borrows an escape table for the single byte that follows, and the engine reverts to the text table on its own.

```

h_print                                ' most bytes: emit the char
      WRBYTE   pa, scriptr             ' PA holds the current byte
      _ret_    ADD     scriptr, #1

h_esc                                  ' $1B: arm the escape table
      _ret_    SETQ2   #esc_mode       ' next byte only, then reverts

```

Again, what matters is what is *absent*. A hand-written terminal keeps an “am I in an escape?” flag and branches on it for every byte; here there is no flag and no branch. ESC shifts the machine for exactly one dispatch, and normal text resumes with no code to put the state back. A multi-byte sequence (ESC [3 1 m) chains the same trick — each state’s handler arms the table for the next byte — so the parser *is* its table set, not a tangle of conditionals.

TIP

The reverting one-shot is the whole reason this stays clean. If a sequence needs to hold an alternate table across several bytes, arm it with persistent SETQ on the way in and re-arm the base table on the way out — SETQ2 for a one-byte shift, SETQ for a mode you stay in.

18.4 A MIDI dispatcher — the byte as data

MIDI is a byte stream whose **status byte** packs two fields: the high nibble is the command (\$9_n_ = Note On, \$8_n_ = Note Off, \$B_n_ = Control Change, ...) and the low nibble is the channel. XBYTE hands the whole status byte to the handler in PA, so the command *selects* the handler while the channel *rides along* as data in the same byte:

```

h_note_on                                ' $9n: Note On, channel n
      MOV      chan, pa
      AND      chan, #$0f               ' channel is PA[3:0]
      RFBYTE   note                     ' data byte 1: note number
      RFBYTE   vel                      ' data byte 2: velocity
      _ret_    CALL   #voice_on         ' start the voice, return

```

This is “the byte is both the selector and an operand” in its cleanest form. Because the seven channel-voice commands live in the *high* nibble, the alternate high-bit index of a 16-entry table

(§9.2) dispatches straight on the command with the channel falling out in PA[3:0] — sixteen entries cover every voice message, and the channel never costs a fetch. Running status (a data byte arriving with no fresh status byte, meaning “same command as last time”) needs a little more: the handler remembers the current command and routes a data-valued byte back into it. That is not free — but it is small, and the FIFO’s self-advancing read keeps the note/velocity pairs aligned.

18.5 A display list — the stream as a movable cursor

A **display list** is a stream of drawing commands — set a color, move the pen, draw a run — that a renderer walks once per frame. Each command byte selects a primitive; its parameters follow inline in the stream, pulled with the FIFO reads of Chapter 5. And because the FIFO position is the list cursor (§10.3), a “repeat” or “jump” command is nothing but an RFAST to a new address — the very mechanism a guest CPU’s branch used in Chapter 15, here doing ordinary graphics:

Here is the whole thing — the complete **non-interpreter** build, and the counterpart to the VM of Chapter 12. It compiles, and it exercises every asset the engine has:

```

CON
    _clkfreq = 200_000_000

    FB_W      = 64                ' framebuffer width, pixels
    FB_H      = 32                ' framebuffer height, pixels

DAT
    ORG

    ' ---- start the display-list engine -----
start
    SETQ2     #5-1                ' load 5 longs into LUT $000..
    RDLONG    $000, ##cmd_table   ' the command dispatch table
    RDFAST    #0, ##displaylist   ' FIFO -> the display list
    PUSH      #$1ff               ' return target per command
    _ret_     SETQ                #0                ' arm: table @ LUT $000, F=0

    ' XBYTE now walks the list. Control reaches 'done' only via END.
done
    JMP       #done               ' park the cog

    ' ---- command handlers (cog-resident) -----
h_end        JMP                #done               ' $00: leave the engine

h_color _ret_ RFVAR              pencol             ' $01: inline colour

h_moveto
    RFWORD    penx
    _ret_     RFWORD              peny

h_hline
    RFVAR     len                 ' $03: paint a horizontal run
    ' inline run length

```

continues on next page →

↪ continued from previous page

```

                MOV     addr, peny          ' addr = fb + y*FB_W + x
                MUL     addr, #FB_W
                ADD     addr, penx
                ADD     addr, fbase
                ADD     penx, len           ' pen ends past the run
                TJZ     len, #.empty       ' zero-length run: nothing to do
.px             WRBYTE  pencol, addr       ' paint one pixel
                ADD     addr, #1
                DJNZ    len, #.px
.empty         RET                        ' run complete

h_repeat
                DJNZ    reps, #.rewind     ' $04: walk the list again
                RET                        ' any passes left?
                RET                        ' no - fall through to END
.rewind_ret_   RDFAST  #0, ##displaylist ' yes - rewind the cursor

' ---- cog variables -----
fbase          LONG    fb                ' framebuffer base (hub)
reps           LONG    3                 ' draw the whole list 3 times
pencol         RES     1
penx           RES     1
peny           RES     1
len            RES     1
addr           RES     1

' ---- hub data: table, display list, framebuffer -----
                ORGH
cmd_table      LONG    h_end              ' $00 -> END
                LONG    h_color           ' $01 -> COLOR
                LONG    h_moveto          ' $02 -> MOVETO
                LONG    h_hline           ' $03 -> HLINE
                LONG    h_repeat          ' $04 -> REPEAT

displaylist    BYTE    $01, $0F          ' COLOR 15
                BYTE    $02, 4,0, 2,0    ' MOVETO 4,2
                BYTE    $03, 20           ' HLINE 20
                BYTE    $02, 4,0, 3,0    ' MOVETO 4,3
                BYTE    $03, 20           ' HLINE 20
                BYTE    $04              ' REPEAT (3 passes)
                BYTE    $00              ' END

fb             BYTE    0[FB_W * FB_H]    ' the framebuffer

```

Five commands, five handlers, and the engine walks a stream of *drawing instructions* with exactly the machinery a language gets. Note what each handler leans on:

- **h_color** and **h_moveto** pull their parameters straight out of the stream with the FIFO reads of Chapter 5 — `RFVAR` for a small value, `RWORD` for a coordinate. The read cursor advances itself, so the next dispatch lands correctly with no bookkeeping.
- **h_hline** does real work, and shows that a handler is just PASM2 — nothing about it is XBYTE-specific.

- **h_repeat** is the interesting one. It re-points the FIFO at the top of the list, and that single RDFAST is the loop. The read cursor is a free, movable program counter (§10.3) — the very same mechanism a guest CPU’s branch used in Chapter 15, here doing ordinary graphics.
- **h_end** leaves the engine by simply not returning to \$1FF (§10.4).

The command stream is *data you emit*, not a program you compile — a scene, a sprite list, a UI layout — yet the engine walks it with the same auto-fetch and dispatch a language gets. This is the mental shift the chapter turns on: XBYTE runs bytecode, and a display list is just bytecode whose “instructions” draw.

HARDWARE

The FIFO reads (RWORD, RVAR) that pull a command’s parameters advance the same read cursor the next dispatch fetches from, so operations and their operands interleave in one stream and the read position takes care of itself — exactly as it did for inline VM operands (§5.3) and guest-CPU operand bytes (§15.1). One mechanism, three very different uses.

18.6 SETQ2 is a general mode switch

Chapter 17 introduced one-shot SETQ2 through the 6809’s prefix pages, where it can read as a CPU-emulation trick. It is neither a trick nor about CPUs. SETQ2 is the general operation “*dispatch the next byte through a different table, then restore mine*” — and every escape or mode shift in a byte stream is an instance of it:

- the **6809** \$10 / \$11 prefix pages (Chapter 17),
- an **ANSI terminal**’s ESC (§18.3),
- **MIDI** System Exclusive, where \$F0 opens a manufacturer data block a different table consumes,
- the **Z80** CB / ED and **x86** \$0F (286+) escape prefixes (§17.1).

It is also how Parallax’s own **Spin2 interpreter** works internally: the interpreter uses one-shot SETQ2 to reach a whole family of *variable-operator* bytecodes through an alternate table, then reverts — the persistent table never has to hold room for them. When you meet a stream where “the next thing means something different,” SETQ2 is the answer, and its automatic revert (§8.3) is what makes the shift free.

18.7 When XBYTE is the wrong tool

The engine is not free. Some problems are stream-shaped and still do not want it, and this section is the honest list.

Three of them are disqualifying — they are not a matter of degree. If any one holds, the engine is simply unavailable to you, and no amount of wanting it will help:

The disqualifier	Why	What you do instead
The stream is not in hub RAM	The FIFO reads hub, and only hub . If your data lives in external PSRAM or HyperRAM, or arrives live from a pin, auto-fetch cannot reach it (§13.3)	write the fetch yourself, keep EXECF dispatch — rung 2
LUT is not free	XBYTE reads its table from LUT . If a palette, a line buffer, or a prefetch queue has already claimed it, the table must live in hub — and the engine cannot read a table in hub	keep the table in hub, dispatch with RDLONG + EXECF
You need cross-cutting work per symbol	XBYTE’s loop is hardware ; there is no loop body (§13.4). Cycle pacing, progress counters, timeout checks, tracing — none of it has anywhere to live	use the software loop (§6.4). Three clocks, and you get a place to stand

The rest are a matter of degree — the engine will work, it just will not earn its keep:

The situation	Why XBYTE does not pay	Use instead
There is no ordered stream to walk	auto-fetch has nothing to fetch	a plain loop over the data
The stream has one fixed record format	the many-way table sits idle; you are using auto-fetch only	RDFAST + the RF reads directly, no engine
Each symbol maps straight to output, no logic	dispatch is just a lookup	one RDLUT per byte is cheaper than arming the engine
The state changes on nearly every symbol	constant re-arming outweighs the saved dispatch	a conventional RDLUT-plus-branch state loop
The unit is a fixed-width word , not a byte	byte auto-fetch does not apply — the RISC case (§14.2)	read the word, dispatch on an extracted field — or see below

HARDWARE

One more option: do not interpret at all.

For a fixed-width RISC guest — ARM, MIPS, RISC-V — there is a strategy that beats every rung of the ladder: **translate the guest’s instructions into native PASM2 once, and then just run them**. A just-in-time translator does the decode a single time per instruction *ever*, instead of once per *execution*, and a hot loop then runs at native P2 speed with no dispatch at all.

This is not hypothetical; it is what the P2’s RISC-V implementation does. It is a substantially bigger undertaking than an interpreter, and it is the right answer when the guest’s instructions are regular enough to translate and hot enough to be worth translating.

XBYTE is an interpreter engine; interpretation is not always the goal.

The through-line is Chapter 13’s three decisions, generalised beyond CPUs: **XBYTE pays when your data is in hub, your LUT is free, you have no per-symbol cross-cutting work, and the byte genuinely selects one of many behaviours.** Weaken any of those and a simpler loop will match it — without spending 256 longs of LUT on a table.

TIP

A quick decision rule: if you can describe the job as *“read a byte from hub, pick one of many things to do, repeat”* — and the pick is not a trivial lookup, and you have nothing to do *between* the picks — XBYTE fits. The more the byte doubles as data, the table doubles as state, or the read cursor moves, the better it fits.

Part V: Reference

This part is for lookup. Chapter 19 is the per-instruction reference for everything XBYTE is built from; Chapter 20 collects the configuration values — the mode-operand layout, the registers, and the memory ranges — in one place. The appendices that follow add quick-reference cards, the encoding summary, pointers to community implementations, and troubleshooting.

Chapter 19: Instruction Reference

The instructions XBYTE uses, grouped by role. Encodings are given in the P2's EEEE form (the leading EEEE is the condition field). All are 2-clock instructions except **EXECF** (4 clocks).

19.1 The skip family

Instruction	Syntax	Encoding	Effect
SKIP	SKIP $\{\#\}D$	EEEE 1101011 00L DDDDDDDD 000110001	Cancel each of the next up-to-32 instructions whose bit in D is set; cancelled instructions still consume their clocks. Works in cog, LUT, and hub. No flag effect.
SKIPF	SKIPF $\{\#\}D$	EEEE 1101011 00L DDDDDDDD 000110010	Fast skip: the PC leaps over each of the next up-to-32 instructions whose bit in D is set; skipped instructions cost nothing. (Under XBYTE the pattern comes from EXECF and is 22 bits — §4.3.) Cog/LUT only. No flag effect.
EXECF	EXECF $\{\#\}D$	EEEE 1101011 00L DDDDDDDD 000110011	Jump to D[9:0] in cog/LUT, then apply D[31:10] as a SKIPF pattern. PC = {10'b0, D[9:0]}. The dispatch vehicle. No flag effect. 4 clocks.

19.2 Arming

Instruction	Syntax	Encoding	Effect
SETQ	SETQ {#}D	EEEE 1101011 00L DDDDDDDDDD 000101000	Load D as the persistent XBYTE mode operand (with <code>_RET_</code> and <code>\$1FF</code> on the stack, starts/continues the engine). Outside XBYTE, sets Q for block transfers.
SETQ2	SETQ2 {#}D	EEEE 1101011 00L DDDDDDDDDD 000101001	Load D as a one-shot XBYTE mode operand — applies to the next bytecode only, then reverts to the last SETQ mode. Outside XBYTE, sets Q for LUT block transfers.

19.3 The FIFO bytecode stream

Instruction	Syntax	Encoding	Effect
RDFAST	RDFAST {#}D, {#}S	EEEE 1100011 1LI DDDDDDDDDD SSSSSSSSSS	Begin a fast sequential hub FIFO read at S[19:0]; D[13:0] = block size in 64-byte units (0 = unlimited), D[31] = no-wait. Precedes all RFxxxx reads.
RFBYTE	RFBYTE D {WC/WZ/WCZ}	EEEE 1101011 CZ0 DDDDDDDDDD 000010000	Read a zero-extended byte from the FIFO into D. C = byte MSB, Z if zero. Fetches the bytecode.
RFWORD	RFWORD D {WC/WZ/WCZ}	EEEE 1101011 CZ0 DDDDDDDDDD 000010001	Read a zero-extended word from the FIFO — a fixed 16-bit inline operand.
RFLONG	RFLONG D {WC/WZ/WCZ}	EEEE 1101011 CZ0 DDDDDDDDDD 000010010	Read a long from the FIFO — a fixed 32-bit inline operand.
RFVAR	RFVAR D {WC/WZ/WCZ}	EEEE 1101011 CZ0 DDDDDDDDDD 000010011	Read a zero-extended 1–4-byte variable-length value into D. C = 0.
RFVARS	RFVARS D {WC/WZ/WCZ}	EEEE 1101011 CZ0 DDDDDDDDDD 000010100	Read a sign-extended 1–4-byte variable-length value into D. C = value MSB.
GETPTR	GETPTR D	EEEE 1101011 000 DDDDDDDDDD 000110100	Get the current FIFO hub pointer into D. XBYTE writes this to PB each dispatch.

Chapter 20: Configuration Constants & Patterns

20.1 The mode operand

The value handed to SETQ/SETQ2, written %A...F:

- **A** (high bits) — the LUT base address of the dispatch table; the number of A bits grows as the table shrinks (§9.2).
- **middle pattern** — selects table size (256/128/64/32/16) and, in the 256 case, compression (§9.2–9.3).
- **F** (bit 0) — flag write: F=1 writes $C \leftarrow \text{index bit 1}$, $Z \leftarrow \text{index bit 0}$ each dispatch; F=0 leaves flags alone (§9.4).

Goal	Operand
256-entry table at LUT \$100, flags off	\$100
256-entry table at LUT \$000, flags off	\$0
256-entry table, flags on	base, with bit 0 = 1
256 with 16-primary compression, threshold B	%ABBBB00xF (§9.3)
smaller tables	per the §9.2 patterns

20.2 Registers and ranges

Item	Value
PA	\$1F6 — current bytecode (written clock 2); usable as an immediate operand in the handler
PB	\$1F7 — current FIFO read pointer (written clock 5)
Return target on stack	\$1FF — what each bytecode routine returns to, triggering the next dispatch
Handler address range	cog \$000–\$1FF, LUT \$200–\$3FF (EXECF jumps to a 10-bit cog/LUT address)
LUT entry format	[9:0] = handler address, [31:10] = 22-bit SKIPF pattern
Hardware stack depth	8 levels
Dispatch overhead	6 clocks/bytecode; minimum loop 8 clocks

20.3 The arming pattern

```
                SETQ2    #N-1                ' load N-long table into LUT
                RDLONG   $000, ##table        ' (or the chosen LUT base)
                RDFAST   #0, ##program        ' FIFO -> bytecode stream
                PUSH     # $1ff               ' return target per bytecode
_ret_          SETQ     #mode                ' arm persistent mode = start
```

Part VI: Appendices

The appendices are lookup material and pointers: the quick-reference cards, the encoding summary, community implementations to study, and a troubleshooting guide.

Appendix A: XBYTE Quick Reference

The dispatch cycle (8 clocks, 6 overhead):

Clk	Activity
1	RFBYTE bytecode · SKIPF #0 (overlaps prior _RET_)
2	MOV PA, bytecode · RDLUT
3	RDLUT → D
4	EXECF D begin
5	MOV PB, GETPTR · MODCZ (if F) · EXECF branch
6–7	pipeline flush + reload
8	first handler instruction; _RET_ → loop to clock 1

Table sizes (mode operand %A...F):

Size	Operand	Base	Index	Alt index
256	%A0000000xF	%A000000000	b[7:0]	—
256+compress	%ABBBB00xF	%A000000000	b[7:4]<B → b[7:0]; else group	—
128	%AAxx0010F	%AA00000000	b[6:0]	%AAxx0011F → b[7:1]
64	%AAAx1010F	%AAA0000000	b[5:0]	%AAAx1011F → b[7:2]
32	%AAAAx100F	%AAAA000000	b[4:0]	%AAAAx101F → b[7:3]
16	%AAAAA110F	%AAAAA00000	b[3:0]	%AAAAA111F → b[7:4]

F bit: 0 = flags untouched; 1 = C ← index bit 1, Z ← index bit 0.

Bit 1 selects the index form: 0 = low-bits index (primary), 1 = high-bits index (the *Alt index* column). Ignored in 256 mode. Leave **0** unless you want the alternate form (§8.2).

Arming checklist: load table → LUT · RFAST the stream · PUSH #\$1FF · _RET_ SETQ #mode. (A CALL will *not* substitute for the PUSH — it pushes its own return address, not \$1FF.)

Appendix B: Instruction Encoding Summary

Instruction	Encoding	Clocks	Flags
SKIP {#}D	EEEE 1101011 00L DDDDDDDDD 000110001	2	—
SKIPF {#}D	EEEE 1101011 00L DDDDDDDDD 000110010	2	—
EXECF {#}D	EEEE 1101011 00L DDDDDDDDD 000110011	4	—
SETQ {#}D	EEEE 1101011 00L DDDDDDDDD 000101000	2	—
SETQ2 {#}D	EEEE 1101011 00L DDDDDDDDD 000101001	2	—
RDFAST {#}D, {#}S	EEEE 1100011 1LI DDDDDDDDD SSSSSSSSS	2	—
RFBYTE D {WC/WZ/WCZ}	EEEE 1101011 CZ0 DDDDDDDDD 000010000	2	C=MSB, Z
RFVAR D {WC/WZ/WCZ}	EEEE 1101011 CZ0 DDDDDDDDD 000010011	2	C=0, Z
RFVARS D {WC/WZ/WCZ}	EEEE 1101011 CZ0 DDDDDDDDD 000010100	2	C=MSB, Z
GETPTR D	EEEE 1101011 000 DDDDDDDDD 000110100	2	—

Appendix C: Further Implementations

The P2 community has built real interpreters and CPU emulators that run on physical silicon. These are pointers for further study, and the implementations the rung assignments in Chapters 13 and 14 are drawn from. (Where those chapters cite a specific behaviour, it comes from one of these; where a project’s dispatch is not publicly documented, the entry says so.)

The useful thing to read from this table is **which rung of the dispatch ladder (§13.2) each one stands on** — the pattern is not the obvious one:

Guest	Rung	Fetch	Dispatch
Spin2 bytecode (<i>the reference</i>)	3 — XBYTE	auto-fetch	XBYTE
ZPU	3 — XBYTE	auto-fetch	XBYTE
Intel 8080	3 — XBYTE	auto-fetch	XBYTE
Zilog Z80	2	hand-rolled	LUT + EXECF
65816	2	hand-rolled (PSRAM queue)	hub table + EXECF
Motorola 68000	2	hand-rolled	nibble table + patched JMP
Intel 8086	2	hand-rolled	EXECF — <i>and</i> , in a second implementation, a plain JMP
RISC-V	—	—	JIT to native PASM2

The pattern in that table: nearly every emulator keeps the dispatch asset (EXECF plus a table), while only the small, hub-resident guests also keep auto-fetch — and a fixed-width RISC guest skips interpretation altogether (§C.7).

C.1 The reference: Parallax’s own XBYTE

Two pieces of first-party code are the best primary sources here.

- **The XBYTE demo** — in Parallax’s `propeller` repository on GitHub (<https://github.com/parallaxinc/propeller>), at `resources/FPGA Examples/xbyte.spin2`. About sixty lines: it loads a table, primes the FIFO, arms the engine, and runs five bytecodes. It also carries Parallax’s own clock-by-clock account of the dispatch cycle, which is the source for Chapter 7.
- **The Spin2 interpreter** — the language’s own bytecode engine, and the most complete XBYTE program available to study. It is where the compression mode, the F bit, and one-shot SETQ2-as-grammar (§17.4) are all put to full use.

C.2 P2 Arc8de — eight 8080 arcade machines on one P2

A single P2-EC module emulating the **Intel 8080** on **all eight cogs at once** — one cog per cabinet, each running an 8080 emulator that executes original arcade ROM code while also generating video (one pin), audio (one pin), and reading five buttons (one pin). Eight independent mini arcade cabinets driven in parallel from one chip — “eight instances of Space Invaders, at once.”

A P2 Forum community project (begun 2020), built by **Coley, Baggers, Chip, and VonSzarvas** with support from Parallax; write-up by **Ken Gracey**. Its dispatch mechanism is not documented in the public materials, so it appears here as an existence proof, not a mined technique.

- **Link:** Parallax project page — <https://www.parallax.com/p2arc8de-one-p2-ec-module-provides-audio-video-and-buttons-for-eight-8-concurrent-games/>

C.3 8080 games emulators — XBYTE in production

The P2 Space Invaders / “Species” emulators run 8080 arcade ROMs, and they are the clearest example of **rung 3** outside the Spin2 interpreter. The 8080’s 64 KB address space fits in hub, so the guest’s code can be streamed — and it is.

They are also where three techniques in this book were found: the **guest-interrupt injection** of §16.4, the **halt-and-back-the-stream-up** idiom of §16.5, and — usefully — the **de-arm-and-substitute** DEBUG technique of §11.3, which appears in the source as a commented-out arming pair beside a hand-rolled loop carrying a `DEBUG()`.

C.4 The “Yume” emulator suite — console emulators on P2 + PSRAM

A family of console emulators by **wuerfel_21** (GitHub organization **IRQsome**; also mirrored on SourceHut as `~wuerfel_21`). Each runs console ROM images on a P2 with external PSRAM.

Project	Console	Guest CPU(s)	Repository	Status
MegaYume	Sega Mega Drive / Genesis	Motorola 68000 + Z80	https://github.com/IRQsome/MegaYume	released
NeoYume	SNK Neo Geo AES	Motorola 68000 + Z80	https://github.com/IRQsome/NeoYume	released
MisoYume	Super Nintendo (SNES)	65(C)816	https://github.com/IRQsome/MisoYume	beta

These use no XBYTE at all — the appendix’s sharpest illustration that instruction shape does not decide the rung. They keep `EXEC/SKIPF` dispatch and write their own fetch, for the reasons Chapter 13 sets out: a console ROM is megabytes and lives in PSRAM, which the FIFO cannot

reach; LUT is wanted for other things; and cycle-accurate emulation needs a loop body to pace in.

MisoYume makes the point sharply: the 65816 is byte-stream and opcode-first — by instruction shape the *ideal* XBYTE guest — and it takes rung 2 anyway. Read **MegaYume’s Z80 core** for the dispatch loop that does bus arbitration, cycle pacing, and a refresh register between every guest instruction (§13.4), and for the two-level nibble dispatch its 68000 uses (§14.2).

C.5 Intel 8086 — the same guest, more than one way

The P2 has more than one 8086 emulator, and together they are the clearest demonstration that **dispatch is a ladder, not a switch** (§13.2). **Marco Maccaferri** wrote two. His `simple_i8086` reads each opcode and EXECFs through a table with SKIP patterns — rung 2 with the dispatch asset. His `i8086_xt` — a complete IBM PC XT with BIOS, CGA, and BASIC — instead reads the opcode and takes a plain JMP through a table of bare addresses. Same guest processor, two rungs, one author.

`i8086_xt` also ships in **two variants — guest memory in hub, and guest memory in PSRAM** — and diffing that pair is the demonstration behind §13.1: the memory backend changes completely (about a hundred lines out of more than eight thousand) and **the dispatch does not move at all**.

Both are presented in the *Intel 8086 CPU Emulator* thread on the Parallax forums (<https://forums.parallax.com/discussion/174634/intel-8086-cpu-emulator>).

C.6 Zog — the ZPU

A **ZPU** (zero-operand stack machine) interpreter — originally by *heater*, with a P2 port maintained by **totalspectrum** (Eric Smith): <https://github.com/totalspectrum/zog>. The ZPU is a byte-opcode stack machine whose memory image fits comfortably in hub, which puts it squarely at **rung 3** — and it arms XBYTE exactly as Chapter 8 describes.

C.7 riscvemu — the road not taken

A **RISC-V** emulator for the Propeller by **totalspectrum**: <https://github.com/totalspectrum/riscvemu>. It is here because of what it *does not* do: rather than interpret 32-bit fixed-width instructions, it **translates them to native PASM2** and runs the translation (§18.7). For a regular, fixed-width guest, a JIT beats every rung of the ladder, and riscvemu is the P2 example.

C.8 A minimal community example — the “essential XBYTE” VM

Everything above is a production interpreter or emulator. This last pointer is the opposite, and just as useful: the **smallest complete XBYTE VM** a community member could reduce it to — four

bytecodes (PUSH, ADD, SUB, HALT) that compute a value and blink it on an LED. It is worth reading for two things this book otherwise only describes:

- it runs the engine in a **dedicated cog**, launched with `COGINIT` and driven by a three-long hub **mailbox** (the §12.4 pattern), and
- its halt handler **pops the arming \$1FF** before returning the cog to idle, so the cog can be armed again for the next job (the §10.4 re-arm rule, made concrete).

It appears in the Parallax forum thread “**basic XBYTE questions**” — <https://forums.parallax.com/discussion/176253/basic-xbyte-questions> — posted by **refaQtor**, where **Eric Smith (ersmith)** and **Christof Eb.** work through the same table-in-LUT, arming, and hub-versus-LUT points this book makes. Read it as a compact worked example, not a specification: it is community code, and the authority for everything it does is the *Parallax Propeller 2 Documentation v35* and the chapters here.

Appendix D: Troubleshooting

When the engine misbehaves, the cause is almost always one of a handful of arming, table, or timing mistakes. Match the symptom below to its likely cause, the fix, and the section that explains it.

Symptom	Likely cause	Fix
Engine never dispatches	no \$1FF on the stack before the arming <code>_RET_</code>	PUSH <code>#\$1FF</code> immediately before <code>_RET_ SETQ</code> (§8.1)
First bytecode is garbage	FIFO not primed	run RDFSFAST on the bytecode stream before arming (§5.1)
Wrong handler runs	address/skip fields transposed in a table entry	address in [9:0], SKIPF pattern in [31:10] (§6.1)
Handler runs the wrong variant	wrong SKIPF pattern in the table entry	recompute the pattern; remember every set bit <i>skips</i> (§4.4)
Dispatch corrupts after a few bytecodes	hardware stack overflow	shorten handler call chains; the stack is 8 levels (§10.1)
A reusable interpreter cog drifts or faults after several <i>jobs</i>	each arming left its \$1FF on the stack — the exit never popped it	POP the \$1FF as the halt handler exits, before re-arming (§8.1, §10.4)
Flags behave unexpectedly across bytecodes	F bit set when not intended (or vice-versa)	set/clear bit 0 of the mode operand deliberately (§9.4)
A prefixed/extended opcode decodes as the base opcode	no alternate-table handling	make the prefix a one-shot SETQ2 handler (§8.3, §17.2)
A prefix corrupts the <i>next</i> instruction rather than redirecting it	it is a modifier prefix, not a map prefix — SETQ2 is the wrong tool	set a state register and re-fetch (§17.3)
Branch goes nowhere	guest PC changed without re-pointing the FIFO	a branch is a RDFSFAST to the new address (§10.3)
Stream resumes at the wrong place after a hub call	the called code left the FIFO elsewhere	re-point from PB: RDFSFAST <code>#0</code> , PB on the way out (§10.3)
A handler is corrupted intermittently, under load	an interrupt split an atomic sequence — a CORDIC command and its result, or a read-modify-write	fence it with a one-iteration REP (§7.4). <i>This is the bug that will cost you the most time, because it is timing-dependent and will not reproduce</i>
Instructions after a handler get skipped	the handler's SKIPF pattern was longer than the handler and ran on past it	size each pattern to its body; in a hand-rolled loop, add a NOP landing pad (§4.5, §11.3)
SETQ2 did something you did not intend	it does two jobs — block-move count, or one-shot mode — and only the <i>next</i> instruction decides which	look at the instruction after it (§8.3)
Engine works, but you cannot see what it is doing	there is no loop body to instrument	read the state with GETBRK, or de-arm and substitute the software loop (Chapter 11)

Index

- **Addressing-mode matrix** (two-stage dispatch) — §15.4
- **Application map** (other uses) — Ch. 18, §18.2
- **Arming** — Ch. 8; quick card, App. A
- **Auto-fetch** (the asset, and its cost) — §13.1, §13.3
- **Beyond interpreters** (applications) — Ch. 18
- **Bit 10** (spare flag in a table entry) — §4.5
- **Budget** (what XBYTE costs you) — §3.6
- **Bytecode** (definition) — Ch. 3; in PA — Ch. 6, §10.2
- **CALL depth** (skipping suspended) — §4.5, §11.1
- **Compression mode** (%ABBBB) — §9.3; decoded — §9.5
- **Cycle accuracy** (guest) — §14.8, §13.4
- **Debugging XBYTE** — Ch. 11; GETBRK — §11.1; de-arm and substitute — §11.3
- **Decimal mode** (guest BCD) — §14.6
- **Dispatch by hand** (the software loop) — §6.4, §11.3
- **Dispatch cycle** (8 clocks) — Ch. 7; App. A
- **Dispatch ladder** (jump table · EXECF · XBYTE) — §13.2
- **Dispatch table** (LUT) — Ch. 6; building entries — §6.2
- **Display list** (application) — §18.5
- **EXECF** — §4.3; synthesised operand — §16.4; Ch. 19
- **F bit** (flags from bytecode) — §9.4, §9.5, §20.1
- **FIFO** — Ch. 5; RDFSFAST — §5.1; resuming after a hub CALL — §10.3
- **Flags** (guest) — §14.5
- **GETBRK** — §11.1
- **GETPTR / PB** — §5.4, §10.3, §20.2
- **Guest CPU survey** — Ch. 14
- **Hardware stack / \$1FF** — §8.1, §10.1
- **Inline operands** — §5.3, §10.3
- **Interrupts (guest)** — Ch. 16; injecting one — §16.4; halt — §16.5
- **Interrupts (P2)** — §7.4; the REP fence — §7.4
- **JIT** (translate, don't interpret) — §18.7
- **Landing pad** (trailing SKIP pattern) — §4.5, §11.3
- **Loop body** (there isn't one) — §13.4, §3.6
- **LUT entry format** — §6.1, §20.2
- **Memory model** (where the guest's code lives) — §13.1, §13.3, §14.2
- **MIDI dispatcher** (application) — §18.4
- **Mode operand** — §8.2, Ch. 9; a real one, decoded — §9.5; App. A
- **Overhead** (6 clocks) — §3.2, §7.2
- **PA** (current bytecode) — §6.3, §20.2
- **Prefixes** — the two kinds — §17.1; map — §17.2; modifier — §17.3

- **REP** (as an interrupt fence) — §7.4
- **RFBYTE / RFWORD / RFLONG** — §5.2, Ch. 19
- **RFVAR / RFVARS** — §5.3, Ch. 19
- **SETQ / SETQ2** — §8.3, §18.6, Ch. 19; its **two jobs** — §8.3; as grammar — §17.4
- **Shared-handler idiom** — §4.4, §12.3; industrial form — §15.4
- **SKIP** — §4.1, Ch. 19
- **SKIPF** — §4.2, Ch. 19; suspended in a **CALL** — §4.5
- **State machine** (table-as-state) — §18.3, §18.6, §17.4
- **Table sizes** — §9.2; App. A
- **Terminal / ANSI reader** (application) — §18.3
- **Three decisions** (fetch · dispatch · memory) — §13.1
- **When to reach for XBYTE** — §3.5, §3.7, §18.7
- **Bit 1** (index-form select; ignored in 256 mode) — §8.2, §9.2, App. A
- **6502 emulator** — Ch. 15
- **6809 / prefix pages** — §17.2
- **8086 / x86** — §14.2, §17.1, §17.3
- **65816** (byte-stream, but off-chip → rung 2) — §14.2
- **68000** — §14.2
- **Z80** (both kinds of prefix) — §17.1, §14.3